

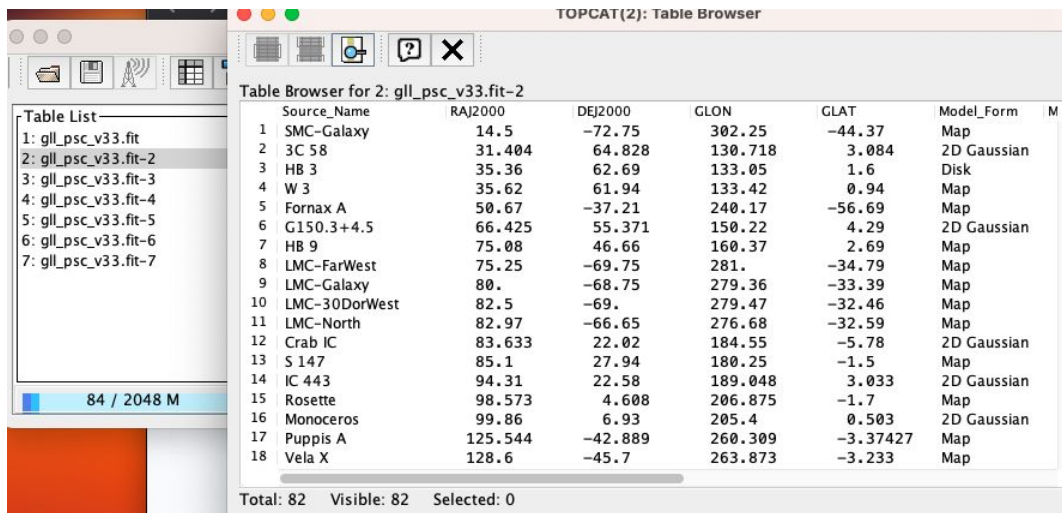
Applying ML/Deep Learning Techniques to Gamma Ray Astronomy

Krishiv Bhatia

Update: 12/08/2023

Update on Progress

- Refreshed Jerry's YouTube presentation
- Downloaded the 4FGL-DR4 FITS file from https://fermi.gsfc.nasa.gov/ssc/data/access/lat/14yr_catalog/
- Installed the TOPCAT tool on my laptop
- Viewed the 4FGL-DR4 FITS file from TOPCAT.



The screenshot shows the TOPCAT(2): Table Browser window. On the left, a 'Table List' sidebar shows several FITS files, with 'gll_psc_v33.fit-2' selected. The main panel displays a table titled 'Table Browser for 2: gll_psc_v33.fit-2'. The table has columns for Source_Name, RAJ2000, DEJ2000, GLON, GLAT, Model_Form, and M. It lists 18 sources, including SMC-Galaxy, 3C 58, HB 3, W 3, Fornax A, G150.3+4.5, HB 9, LMC-FarWest, LMC-Galaxy, LMC-30DorWest, LMC-North, Crab IC, S 147, IC 443, Rosette, Monoceros, Puppis A, and Vela X. At the bottom, it shows 'Total: 82', 'Visible: 82', and 'Selected: 0'.

	Source_Name	RAJ2000	DEJ2000	GLON	GLAT	Model_Form	M
1	SMC-Galaxy	14.5	-72.75	302.25	-44.37	Map	
2	3C 58	31.404	64.828	130.718	3.084	2D Gaussian	
3	HB 3	35.36	62.69	133.05	1.6	Disk	
4	W 3	35.62	61.94	133.42	0.94	Map	
5	Fornax A	50.67	-37.21	240.17	-56.69	Map	
6	G150.3+4.5	66.425	55.371	150.22	4.29	2D Gaussian	
7	HB 9	75.08	46.66	160.37	2.69	Map	
8	LMC-FarWest	75.25	-69.75	281.	-34.79	Map	
9	LMC-Galaxy	80.	-68.75	279.36	-33.39	Map	
10	LMC-30DorWest	82.5	-69.	279.47	-32.46	Map	
11	LMC-North	82.97	-66.65	276.68	-32.59	Map	
12	Crab IC	83.633	22.02	184.55	-5.78	2D Gaussian	
13	S 147	85.1	27.94	180.25	-1.5	Map	
14	IC 443	94.31	22.58	189.048	3.033	2D Gaussian	
15	Rosette	98.573	4.608	206.875	-1.7	Map	
16	Monoceros	99.86	6.93	205.4	0.503	2D Gaussian	
17	Puppis A	125.544	-42.889	260.309	-3.37427	Map	
18	Vela X	128.6	-45.7	263.873	-3.233	Map	

Update on Progress

- Downloaded ATNF Pulsar Catalogue tar gz file. Followed README to produce the executable psrcat file.
- Followed README
- Psrcat -h works but I get error in the db_file command
- What is this for? How to fix error?

```
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % ls
README          evalwrap.c      libpsrcat.a     psrcat.db
defineParams.c  evalwrap.o      makeit          psrcat.h
defineParams.o  externalCall.c  myrefs.bbl     psrcat_ref
displayOutput.c externalCall.o   parseParameters.c readCatalogue.c
displayOutput.o galcoord_ecliptic.c parseParameters.o readCatalogue.o
evaldefs.h      galcoord_ecliptic.o plotParams.c    wc_strncmp.c
evalkern.c      galcoord_ecliptic.o plotParams.o    wc_strncmp.h
evalkern.h      galcoord_ecliptic.o psrcat          wc_strncmp.o
evalkern.o      glitch.db       psrcat.c
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -h
-all          Merge with $PSRCAT_RUNDIR/obs*.db
-allref       Lists the entire bibliographic reference file
-bib           Provide bibliographic reference
-bibliography  Lists references within the catalogue
-c            List of parameters to obtain from the database
-check        Checks the catalogue for incorrect entries
-db_file      Database path and name
-descend      Sort in descending order
-e            Produce short ephemeris files suitable for TEMPO
-e2           Produce long ephemeris files
-e3           Produce ephemeris file with selected parameters
-exactMatch    Require exact match for string comparisons
-h            This help information
-l            Logical string for filtering unwanted pulsars
              e.g. p0 > 5
-listref       Provide bibliographic references at end of table
-merge        Merges multiple catalogue files. For example
              -merge "mycat*.db" will merge the public catalogue
              with all the catalogues with filenames mycat*.db in the
              local directory.
-nohead       Do not print a header at the top of the table
-nonumber     Do not print line numbers in output
-nocand       Do not include pulsar candidates
-nointerim    Do not include interim solutions
-null         Null string for displaying for unknown parameter
```

```
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -db_file ./psrcat.db -c "name p0 dm" -l "p0 > 3"
zsh: illegal hardware instruction psrcat -db_file ./psrcat.db -c "name p0 dm" -l "p0 > 3"
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar %
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar % psrcat -db_file ./psrcat.db
zsh: illegal hardware instruction psrcat -db_file ./psrcat.db
rbhatia@Rajeshs-MacBook-Pro-Display psrcat_tar %
```

Next Steps

- Could I have access to the Python scripts that you used with the 3FGL catalog
- Then, if I understand correctly, I need to reproduce your results using PyTorch?
- And then convert to using Neural Networks for Deep Learning?

Update: 01/08/2024

Progress

- 3FGLReproductionBasicLR Python code pointed to 3FGL Catalog file gll_psc_v16.fit.
- Updated it to point to 4FGL catalog file gll_psc_v33.fit.
- Tested with updated feature name
- Modified it for 1 hidden layer NN and DNN (2 hidden layers NN)

Results

Model	AGN Sensitivity	PSR Sensitivity	Accuracy of Best P Fit
Basic LR (0 hidden layer NN)	0.9724137931034482	1.0	0.9733333333333334
Neural Network (1 hidden layer)	0.9620689655172414	1.0	0.9633333333333334
Deep Neural Network (2 hidden layers)	0.9724137931034482	1.0	0.9733333333333334

Features Used

- Used the following features from columns in 4FGL catalog file gll_psc_v33.fit
 - ["PL_Index", "LP_Index", "PLEC_IndexS", "Variability_Index", "Unc_PL_Flux_Density", "Unc_LP_Flux_Density", "Unc_Energy_Flux100", "Unc_PLEC_Flux_Density", "Unc_Flux1000", "LP_beta", "Frac_Variability", "LP_SigCurv", "Signif_Avg"]
 - Found the corresponding 3FGL columns in the 4FGL list
 - Did some trial and error for other columns and checked model performance
- Column list is in pages 32-33 of Abdollahi_2020_ApJS_247_33 paper

Columns

- Printed out from code

1 DataRelease	21 PL_Flux_Density	41 Unc_PLEC_Exp_Index	61 ASSOC_4FGL
2 RAJ2000	22 Unc_PL_Flux_Density	42 PLEC_SigCurv	62 TEVCAT_FLAG
3 DEJ2000	23 PL_Index	43 PLEC_EPeak	63 ASSOC_TEV
4 GLON	24 Unc_PL_Index	44 Unc_PLEC_EPeak	64 CLASS1
5 GLAT	25 LP_Flux_Density	45 Npred	65 ASSOC1
6 Conf_68_SemiMajor	26 Unc_LP_Flux_Density	46 Flux_Band	66 ASSOC_PROB_BAY
7 Conf_68_SemiMinor	27 LP_Index	47 Unc_Flux_Band	67 ASSOC_PROB_LR
8 Conf_68_PosAng	28 Unc_LP_Index	48 nuFnu_Band	68 RA_Counterpart
9 Conf_95_SemiMajor	29 LP_beta	49 Sqrt_TS_Band	69 DEC_Counterpart
10 Conf_95_SemiMinor	30 Unc_LP_beta	50 Variability_Index	70 Unc_Counterpart
11 Conf_95_PosAng	31 LP_SigCurv	51 Frac_Variability	71 Flags
12 ROI_num	32 LP_EPeak	52 Unc_Frac_Variability	72 ASSOC_FGL
13	33 Unc_LP_EPeak	53 Signif_Peak	73 ASSOC_FHL
Extended_Source_Name	34 PLEC_Flux_Density	54 Flux_Peak	74 ASSOC_GAM1
14 Signif_Avg	35 Unc_PLEC_Flux_Density	55 Unc_Flux_Peak	75 ASSOC_GAM2
15 Pivot_Energy	36 PLEC_IndexS	56 Time_Peak	76 ASSOC_GAM3
16 Flux1000	37 Unc_PLEC_IndexS	57 Peak_Interval	77 CLASS2
17 Unc_Flux1000	38 PLEC_ExpfactorS	58 Flux_History	78 ASSOC2
18 Energy_Flux100	39 Unc_PLEC_ExpfactorS	59 Unc_Flux_History	
19 Unc_Energy_Flux100	40 PLEC_Exp_Index	60 Sqrt_TS_History	
20 SpectrumType			

Feature Transformation

- FGL4Dataset feature transformations

```
def __init__(self, wanted_type_col, fluxindices, wantedtypes, wantedfeaturenames, pointsourcecata):  
    # Create x/input data  
    xdata = []  
    ydata = []  
    # Select sources that are a part of the wanted types  
    for source in pointsourcecatalogue.data:  
        for feature_names in wantedfeaturenames:  
            print("{0} = {1}".format(*args: feature_names, source[feature_names]))  
            if source[wanted_type_col] in wantedtypes:  
                unit = []  
                # Append wanted params. Applied a log transformation to paras with highly skewed distributions.  
                unit.append(float(source[wantedfeaturenames[0]]))  
                unit.append(float(source[wantedfeaturenames[1]]))  
                unit.append(float(source[wantedfeaturenames[2]]))  
                unit.append(float(source[wantedfeaturenames[3]]))  
                unit.append(math.log(source[wantedfeaturenames[4]]))  
                if (source[wantedfeaturenames[4]] > 0) else float(source[wantedfeaturenames[4]])  
                unit.append(math.log(source[wantedfeaturenames[5]]))  
                if (source[wantedfeaturenames[5]] > 0) else float(source[wantedfeaturenames[5]])  
                unit.append(math.log(source[wantedfeaturenames[6]]))  
                if (source[wantedfeaturenames[6]] > 0) else float(source[wantedfeaturenames[6]])  
                unit.append(math.log(source[wantedfeaturenames[7]]))  
                if (source[wantedfeaturenames[7]] > 0) else float(source[wantedfeaturenames[7]])  
                unit.append(math.log(source[wantedfeaturenames[8]]))  
                if (source[wantedfeaturenames[8]] > 0) else float(source[wantedfeaturenames[8]])  
                unit.append(float(source[wantedfeaturenames[9]]))  
                unit.append(float(source[wantedfeaturenames[10]]))  
                unit.append(float(source[wantedfeaturenames[11]]))  
                unit.append(float(source[wantedfeaturenames[12]]))
```

Apply log transformations
to params with highly
skewed distributions

PL_Index = 2.5443360805511475

LP_Index = 2.5455660820007324

PLEC_IndexS = 2.539679765701294

Variability_Index = 25.699934005737305

Unc_PL_Flux_Density = 3.7675866251677195e-14

Unc_LP_Flux_Density = 4.284851688957439e-14

Unc_Energy_Flux100 = 4.238916617889388e-13

Unc_PLEC_Flux_Density = 3.952874743566419e-14

Unc_Flux1000 = 2.748007377206818e-11

LP_beta = -0.009841139428317547

Frac_Variability = 0.5357248187065125

LP_SigCurv = 0.0

Signif_Avg = 8.640633583068848

Neural Networks Configurations Used

- Used

- 0 hidden layer Neural Networks - Linear Regression
- 1 hidden layer NN
- Deep Neural Networks (2 hidden layers)

0 hidden layer NN - Linear Regression

```
Linear(in_features=13, out_features=1, bias=True)
```

NN (1 hidden layer)

```
Sequential(  
  (0): Linear(in_features=13, out_features=60, bias=True)  
  (1): ReLU()  
  (2): Linear(in_features=60, out_features=1, bias=True)  
)
```

DNN (2 hidden layers)

```
Sequential(  
  (0): Linear(in_features=13, out_features=250, bias=True)  
  (1): ReLU()  
  (2): Linear(in_features=250, out_features=40, bias=True)  
  (3): ReLU()  
  (4): Linear(in_features=40, out_features=1, bias=True)  
)
```

Other Changes & Github

Classes

- Separated the large code into classes for code modularity, improvement
- Created separate classes for:
 - **Driver:** 3FGLReproductionBasicLR_v1.py, 4FGLReproductionBasic{LR, NN, DNN}.py,
 - **Feature transformation:** FGL3Dataset.py / FGL4Dataset.py
 - **Models:** LogisticRegression.py, NeuralNetwork.py, DeepNeuralNetwork.py
 - **Utility functions:** utils.py (e.g. run_iteration)

- 3FGLReproductionBasicLR_v1.py
 - FGL3Dataset.py
 - LogisticRegression.py
 - utils.py

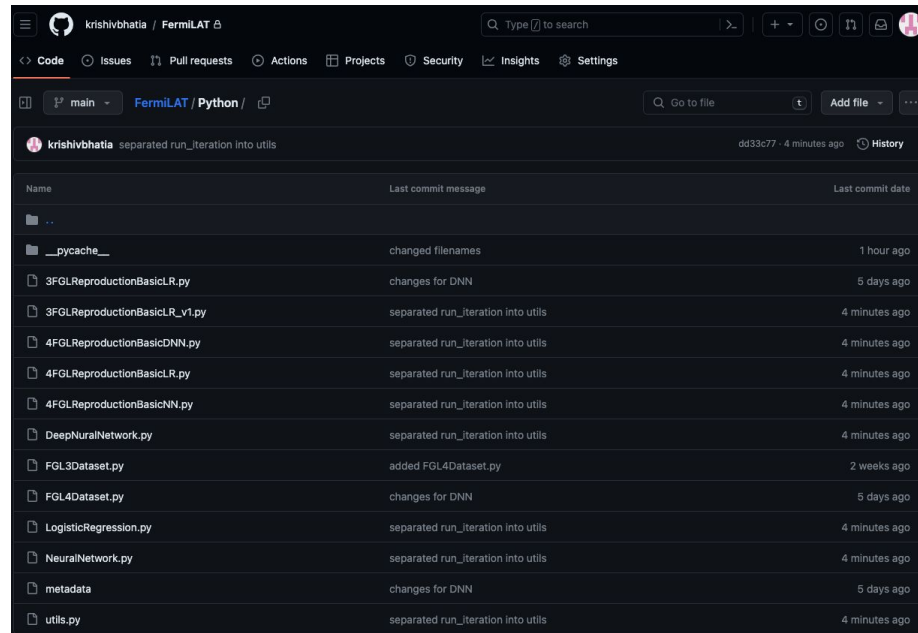
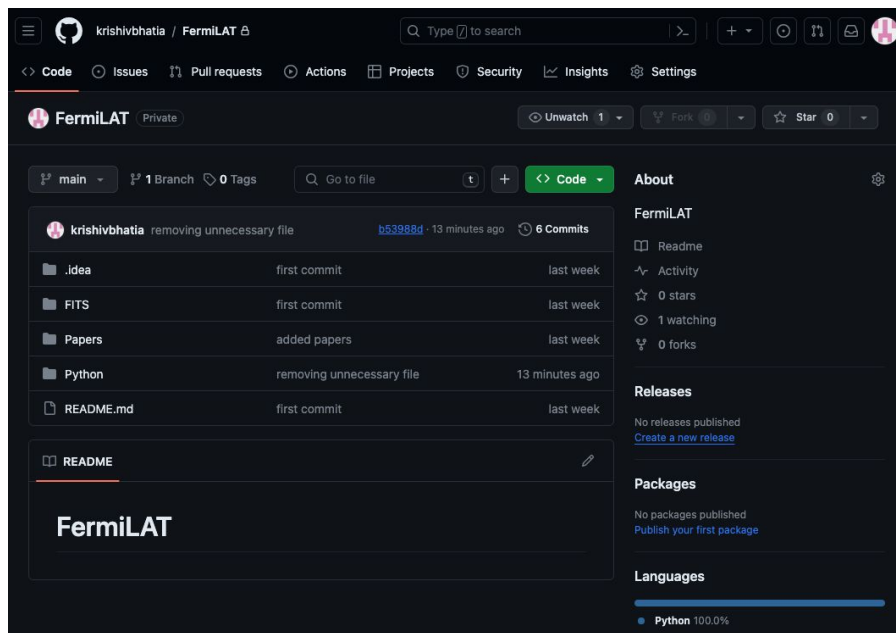
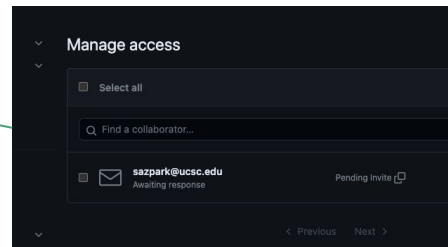
- 4FGLReproductionBasicLR.py
 - FGL4Dataset.py
 - LogisticRegression.py
 - utils.py

- 4FGLReproductionBasicLR.py
 - FGL4Dataset.py
 - LogisticRegression.py
 - utils.py
- 4FGLReproductionBasicDNN.py
 - FGL4Dataset.py
 - DeepNeuralNetwork.py
 - utils.py

Github Repo

- Checked into github
 - <https://github.com/krishivbhatia/FermiLAT/>

Sent you collaborator invite



Results: Linear Regression

Files for Linear Regression

- Python Scripts
 - 4FGLReproductionBasicLR.py
 - FGL4Dataset.py
 - LogisticRegression.py
 - utils.py
- Execution
 - python3 4FGLReproductionBasicLR.py

1-layer NN (No hidden layer)
Basic LR



```
main ▾ FermiLAT / Python / LogisticRegression.py   
vrbhatia changes for DNN  
1 loc) · 1.79 KB  
Blame      
  
import math  
import torch  
import torch.nn as nn  
  
class LogisticRegression(nn.Module):  
    def __init__(self, input_features):  
        print("(len(input_features[0][0]) = ", len(input_features[0][0]))  
        print("input_features[0][0] = ", input_features[0][0])  
        super(LogisticRegression, self).__init__()  
        self.model = nn.Linear(len(input_features[0][0]), 1)  
        print()  
        print(self.model)  
        print()  
  
    def forward(self, x):  
        y_predicted = torch.sigmoid(self.model(x))  
        return y_predicted
```

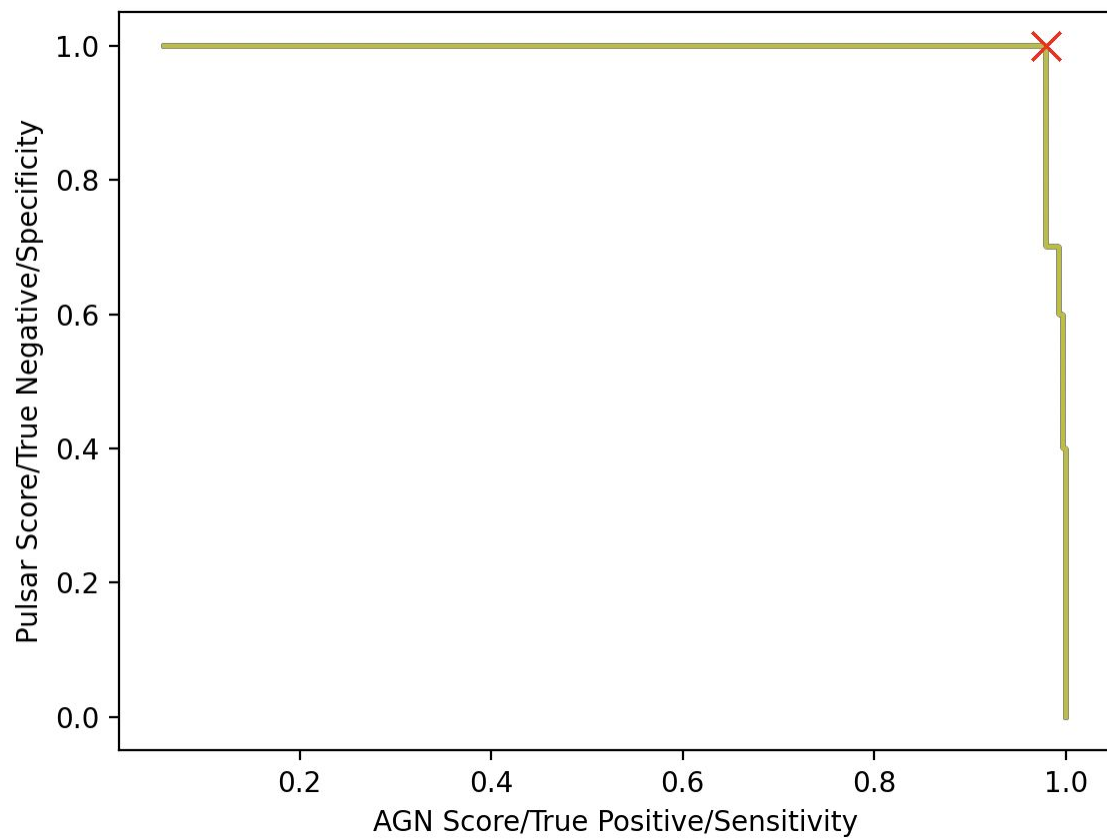
Results

- Linear(in_features=13, out_features=1, bias=True)
- **Results (200 iterations, lr = 0.00001)**
 - optimal_p = 0.9352
 - Best True Positive (AGN Successfully Identified): 282 (290)
 - Best AGN/True Positive/Sensitivity : 0.9724137931034482
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.9733333333333334

Results

- Linear(in_features=13, out_features=1, bias=True)
- **Results (300 iterations, lr = 0.00001)**
 - optimal_p = 0.9352
 - Best True Positive (AGN Successfully Identified): 282 (290)
 - Best AGN/True Positive/Sensitivity : 0.9724137931034482
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.9733333333333334

Plot



Results: NN - 1 hidden layer

Files for NN (1 hidden layer)

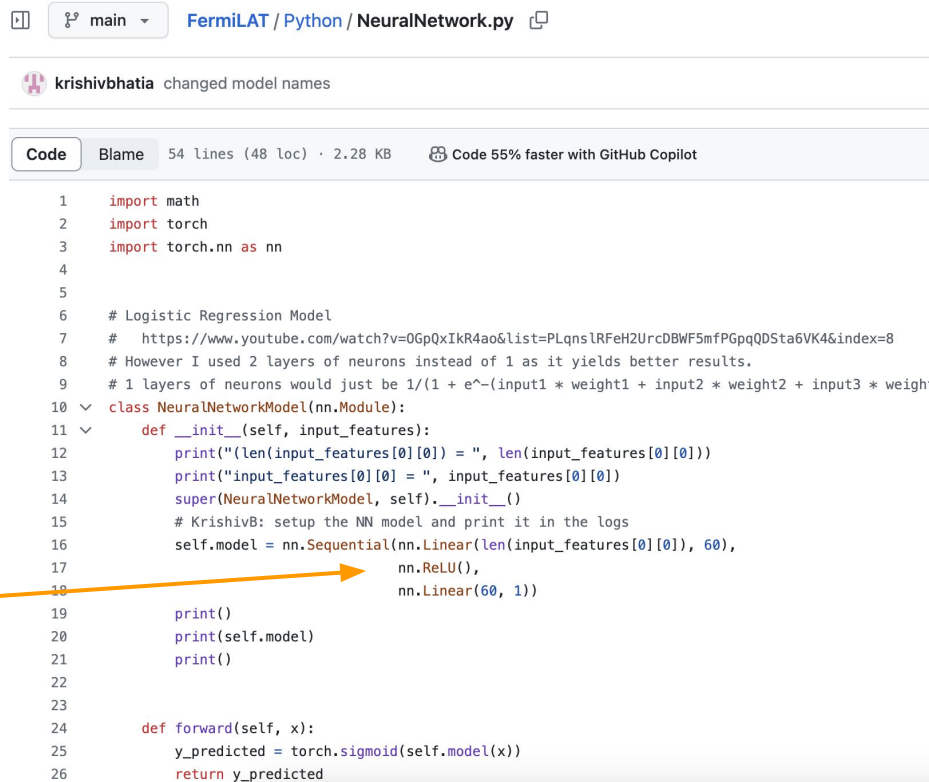
- Python Scripts

- 4FGLReproductionBasicNN.py
- FGL4Dataset.py
- NeuralNetwork.py
- utils.py

- Execution

- python3 4FGLReproductionBasicNN.py

1 hidden layers NN
ReLU activation function



The screenshot shows a GitHub repository for 'FermiLAT / Python / NeuralNetwork.py'. The repository is owned by 'krishivbhatia' and has a commit message 'changed model names'. The file 'NeuralNetwork.py' is selected, showing 54 lines of code (48 loc) and a size of 2.28 KB. The code is written in Python and defines a 'NeuralNetworkModel' class. The class has an 'init' method that sets up the model with two layers: a linear layer with 60 units and a ReLU activation function. The 'forward' method uses 'torch.sigmoid' to return the predicted output. An orange arrow points from the text 'ReLU activation function' to the 'nn.ReLU()' line in the code.

```
1 import math
2 import torch
3 import torch.nn as nn
4
5
6 # Logistic Regression Model
7 # https://www.youtube.com/watch?v=0GpQxIkR4ao&list=PLqns1RFeH2UrcDBWF5mfPGpqQDSta6VK4&index=8
8 # However I used 2 layers of neurons instead of 1 as it yields better results.
9 # 1 layers of neurons would just be 1/(1 + e^~(input1 * weight1 + input2 * weight2 + input3 * weight3))
10 class NeuralNetworkModel(nn.Module):
11     def __init__(self, input_features):
12         print("(len(input_features[0][0]) = ", len(input_features[0][0]))
13         print("input_features[0][0] = ", input_features[0][0])
14         super(NeuralNetworkModel, self).__init__()
15         # KrishivB: setup the NN model and print it in the logs
16         self.model = nn.Sequential(nn.Linear(len(input_features[0][0]), 60),
17                                   nn.ReLU(),
18                                   nn.Linear(60, 1))
19
20         print()
21         print(self.model)
22         print()
23
24     def forward(self, x):
25         y_predicted = torch.sigmoid(self.model(x))
26         return y_predicted
```

Results

- **Model: Sequential(**

(0): Linear(in_features=13, out_features=60, bias=True)

(1): ReLU() ← Activation Function

(2): Linear(in_features=60, out_features=1, bias=True))

1 hidden layer



- **Results (300 iterations, lr = 0.0001)**

- optimal_p = 0.9525
- Best True Positive (AGN Successfully Identified): 279 (290)
- Best AGN/True Positive/Sensitivity : 0.9620689655172414
- Best True Negative (PSR Successfully Identified): 10 (10)
- Best Pulsar/True Negative/Specificity : 1.0
- Accuracy of Best P Fit : 0.9633333333333334

Results

- **Model: Sequential**
 - (0): Linear(in_features=13, out_features=60, bias=True)
 - (1): ReLU()
 - (2): Linear(in_features=60, out_features=1, bias=True))
- **Results (60 iterations, lr = 0.0001)**
 - optimal_p = 0.9525
 - Best True Positive (AGN Successfully Identified): 279 (290)
 - Best AGN/True Positive/Sensitivity : 0.9620689655172414
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.9633333333333334

Results

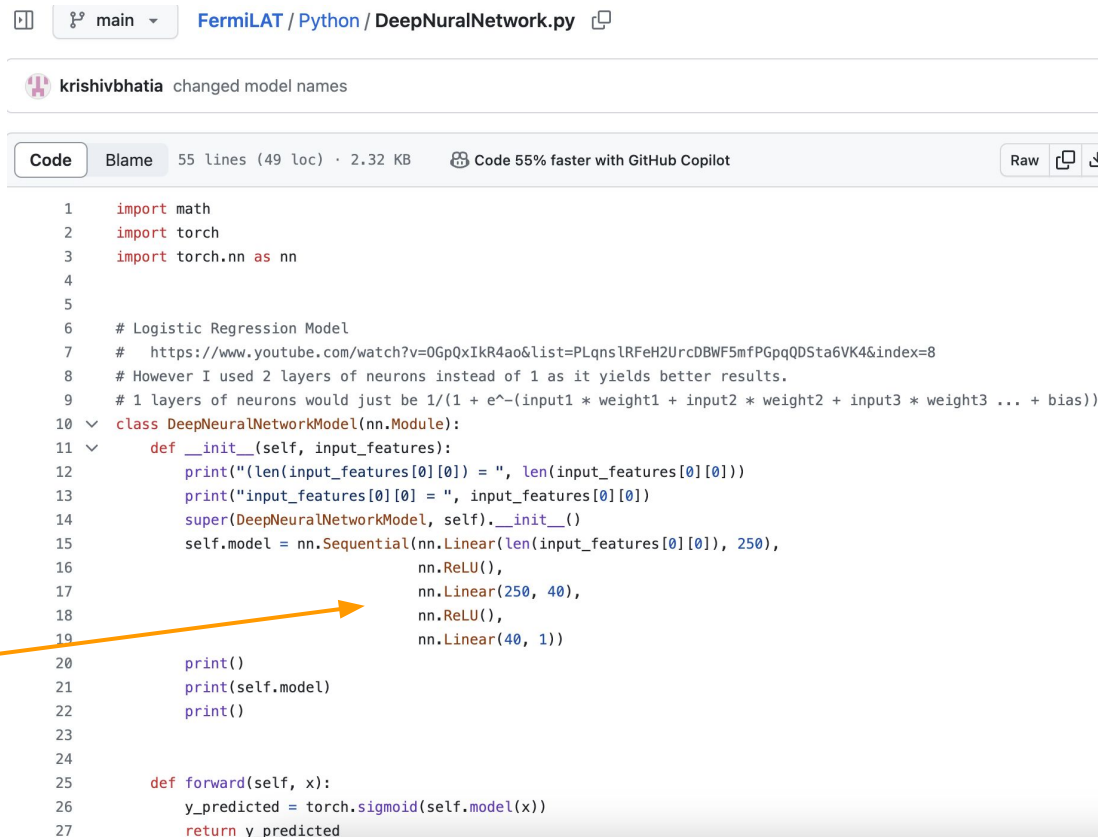
- **Model: Sequential**
 - (0): Linear(in_features=13, out_features=40, bias=True)
 - (1): Tanh() ← tested with tanh() activation function also
 - (2): Linear(in_features=40, out_features=1, bias=True))
- **Results (300 iterations, lr = 0.0001)**
 - optimal_p = 0.9544
 - Best True Positive (AGN Successfully Identified): 278 (290)
 - Best AGN/True Positive/Sensitivity : 0.9586206896551724
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.96

Results: DNN - 2 hidden layers

Files for DNN: 2 hidden layers

- Python Scripts
 - 4FGLReproductionBasicDNN.py
 - FGL4Dataset.py
 - DeepNeuralNetwork.py
 - utils.py
- Execution
 - python3 4FGLReproductionBasicDNN.py

2 hidden layers NN
ReLU activation function



```
1 import math
2 import torch
3 import torch.nn as nn
4
5
6 # Logistic Regression Model
7 # https://www.youtube.com/watch?v=OGpQxIkR4ao&list=PLqnsIRFeH2UrcDBWF5mfPGpqQDSta6VK4&index=8
8 # However I used 2 layers of neurons instead of 1 as it yields better results.
9 # 1 layers of neurons would just be 1/(1 + e^(input1 * weight1 + input2 * weight2 + input3 * weight3 ... + bias))
10 class DeepNeuralNetworkModel(nn.Module):
11     def __init__(self, input_features):
12         print("len(input_features[0][0]) = ", len(input_features[0][0]))
13         print("input_features[0][0] = ", input_features[0][0])
14         super(DeepNeuralNetworkModel, self).__init__()
15         self.model = nn.Sequential(nn.Linear(len(input_features[0][0]), 250),
16                                   nn.ReLU(),
17                                   nn.Linear(250, 40),
18                                   nn.ReLU(),
19                                   nn.Linear(40, 1))
20
21         print()
22         print(self.model)
23         print()
24
25     def forward(self, x):
26         y_predicted = torch.sigmoid(self.model(x))
27         return y_predicted
```

Results

- **Model: Sequential(**

- (0): Linear(in_features=13, out_features=250, bias=True)

- (1): ReLU() ← activation function

- (2): Linear(in_features=250, out_features=40, bias=True)

- (3): ReLU() ← activation function

- (4): Linear(in_features=40, out_features=1, bias=True))

2 hidden layers



- **Results (200 iterations, lr = 0.0001)**

- optimal_p = 0.9475
 - Best True Positive (AGN Successfully Identified): 282 (290)
 - Best AGN/True Positive/Sensitivity : 0.9724137931034482
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.9733333333333334

Results

- **Model: Sequential()**
 - (0): Linear(in_features=13, out_features=250, bias=True)
 - (1): ReLU()
 - (2): Linear(in_features=250, out_features=40, bias=True)
 - (3): ReLU()
 - (4): Linear(in_features=40, out_features=1, bias=True))
- **Results (300 iterations, lr = 0.0001)**
 - optimal_p = 0.9475
 - Best True Positive (AGN Successfully Identified): 282 (290)
 - Best AGN/True Positive/Sensitivity : 0.9724137931034482
 - Best True Negative (PSR Successfully Identified): 10 (10)
 - Best Pulsar/True Negative/Specificity : 1.0
 - Accuracy of Best P Fit : 0.9733333333333334

Questions

- Hardness ratios of LAT energy bands
 - 3FGL Catalog had many bands:
 - "Flux100_300", "Flux300_1000", "Flux1000_3000", "Flux3000_10000", "Flux10000_100000"
 - 4FGL Catalog has only one energy band column present:
 - Energy_Flux100
 - From Abdollahi 2020, definition: *Energy flux from 100MeV to 100GeV obtained by spectral fitting*
 - Where do we get other bands to calculate hardness ratios?

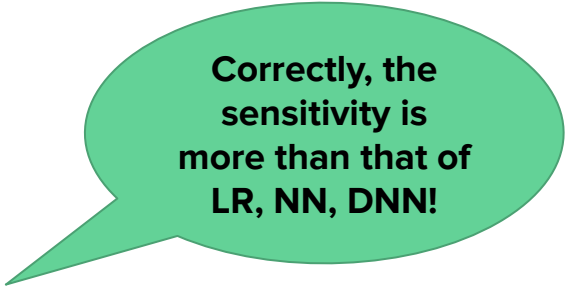
Next Steps

- Random Forest & Decision Tree implementations
 - Found Random Forest & Decision Tree implementations using PyTorch
 - <https://discuss.pytorch.org/t/how-can-i-use-knn-random-forest-models-in-pytorch/42464/2>
 - https://github.com/ValentinFigue/Sklearn_PyTorch
 - Test class shows how to use it:
 - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/random_forest_tests.py
 - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/binary_tree_tests.py
 - Use it to implement Random Forest & Decision Trees in PyTorch for 4FGL Catalog?

Update: 01/15/2024

Progress

- Completed Random Forest & Decision Tree implementations using PyTorch for 4FGL Catalog file
 - Random Forest
 - Highest sensitivity = 98.05749805749805
 - Decision Trees
 - Highest sensitivity = 98.13519813519814



Correctly, the sensitivity is more than that of LR, NN, DNN!

Background

- Referenced Random Forest & Decision Tree implementations using PyTorch
 - https://github.com/ValentinFigue/Sklearn_PyTorch
- Random Forest test class shows how to use it:
 - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/random_forest_tests.py
 - `random_forest = Sklearn_PyTorch.TorchRandomForestRegressor(nb_trees=10, nb_samples=2, max_depth=3)`
 - `random_forest.fit(train_features, train_labels)`
- Decision Tree test class shows how to use it:
 - https://github.com/ValentinFigue/Sklearn_PyTorch/blob/master/tests/binary_tree_tests.py
 - `binary_tree = Sklearn_PyTorch.TorchDecisionTreeClassifier(max_depth=1)`
 - `binary_tree.fit(train_features, train_labels)`

Code Updates

- Checked in code in github for:

Random Forest

- 4FGLReproduction_RF.py
 - FGL4Dataset.py
 - random_forest.py
 - utils.py

Decision Trees

- 4FGLReproduction_decision_tree.py
 - FGL4Dataset.py
 - decision_tree.py
 - decision_node.py
 - utils.py

Github Repo - Python package

 krishivbhatia / FermiLAT 

 |  |  |  |  | 

[Code](#) | [Issues](#) | [Pull requests](#) | [Actions](#) | [Projects](#) | [Security](#) | [Insights](#) | [Settings](#)

  main ▾ [FermiLAT / Python /](#) 

 [Add file ▾](#) [...](#)

 **krishivbhatia** modifications for directories 3adc19f · 4 minutes ago [History](#)

Name	Last commit message	Last commit date
 ..		
 DecisionTrees	modifications for Random Forest and Decsion Trees	4 minutes ago
 LogisticRegression	modifications for Random Forest and Decsion Trees	4 minutes ago
 NeuralNetworks	modifications for Random Forest and Decsion Trees	4 minutes ago
 RandomForests	modifications for Random Forest and Decsion Trees	4 minutes ago
 Utils	modifications for Random Forest and Decsion Trees	4 minutes ago
 <code>__init__.py</code>	modifications for Random Forest and Decsion Trees	4 minutes ago

Github Repo - Random Forests package

 krishivbhatia / FermiLAT 

 |     

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

  main [FermiLAT](#) / [Python](#) / [RandomForests](#) / 

 [Add file](#) 

 **krishivbhatia** modifications for Random Forest and Decsion Trees 5f26b63 · 5 minutes ago [History](#)

Name	Last commit message	Last commit date
 ..		
 4FGLReproduction_RF.py	modifications for Random Forest and Decsion Trees	5 minutes ago
 __init__.py	modifications for Random Forest and Decsion Trees	5 minutes ago
 random_forest.py	modifications for Random Forest and Decsion Trees	5 minutes ago

Github Repo - Decision Trees package

 krishivbhatia / FermiLAT 

 |     

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

  main [FermiLAT](#) / [Python](#) / [DecisionTrees](#) / 

 [Add file](#) 

 **krishivbhatia** modifications for Random Forest and Decsion Trees 5f26b63 · 5 minutes ago [History](#)

Name	Last commit message	Last commit date
 ..		
 4FGLReproduction_DecisionTree.py	modifications for Random Forest and Decsion Trees	5 minutes ago
 __init__.py	modifications for Random Forest and Decsion Trees	5 minutes ago
 decision_node.py	modifications for Random Forest and Decsion Trees	5 minutes ago
 decision_tree.py	modifications for Random Forest and Decsion Trees	5 minutes ago

Github Repo - Utils package

 krishivbhatia / FermiLAT 

Q Type  to search

  main  FermiLAT / Python / Utils / 

Q Go to file  Add file 

 **krishivbhatia** changes in Random Forest and Decision Trees code for FermiLAT 6ebb528 · 9 minutes ago  History

Name	Last commit message	Last commit date
 ..		
 FGL3Dataset.py	modifications for Random Forest and Decsion Trees	4 hours ago
 FGL4Dataset.py	modifications for Random Forest and Decsion Trees	4 hours ago
 __init__.py	modifications for Random Forest and Decsion Trees	4 hours ago
 metadata	modifications for Random Forest and Decsion Trees	4 hours ago
 utils.py	changes in Random Forest and Decision Trees code for FermiLAT	9 minutes ago

Code Details

- Method of creating classifier and invoking fit method:

4FGLReproduction_RF.py

```
# Krishiv Bhatia: Invoke TorchRandomForestClassifier
random_forest = TorchRandomForestClassifier( nb_trees: 200, nb_samples: 2500, max_depth: 15)
print("random forest = ", random_forest.nb_trees, random_forest.nb_samples, random_forest.max_depth)

# Krishiv Bhatia: split train dataset into features and labels
train_features, train_labels = dataset_to_features_labels(train_dataset)
random_forest.fit(torch.FloatTensor(train_features), torch.LongTensor(train_labels))
```

4FGLReproduction_decision_tree.py

```
# Krishiv Bhatia: Invoke TorchDecisionTreeClassifier
decision_tree = TorchDecisionTreeClassifier(20)
print("TorchDecisionTreeClassifier = ", decision_tree.max_depth)

# Krishiv Bhatia: split train dataset into features and labels
train_features, train_labels = dataset_to_features_labels(train_dataset)
decision_tree.fit(torch.FloatTensor(train_features), torch.LongTensor(train_labels))
```

Transforms dataset into
separate feature & labels
arrays (next slide)

Random Forest

Decision Trees

Transformations

- Transformed the tensor feature and labels array into regular arrays

```
main ▾ FermiLAT / Python / Utils / utils.py

Blame 150 lines (126 loc) · 4.75 KB Code 55% faster with GitHub Copilot

def mean(values):
    """
    Mean function.
    """
    m = 0.0
    for value in values:
        m = m + value/len(values)
    return m

def dataset_to_features_labels(dataset):
    # Krishiv Bhatia: train or test dataset is of the form:
    #   tensor array(train/test features), tensor array (output label)
    # or like this:
    #   tensor array(feature 0, feature 1, ... feature 12), tensor array(output label)
    # You can print it like this:
    #   for i in train_dataset:
    #       print(i)
    # An example:
    #   (tensor([ 2.1602,  1.8308,  1.6083, 13.2215, -32.9276, -32.4171, -29.4589,
    #            -32.3813, -24.3150,  0.4812,  0.0000,  2.7429,  5.4350]), tensor([1.]))
    #
    # Convert tensor feature array in dataset to numpy feature array
    train_features = ([i[i[0]].detach().numpy() for i in dataset])
    # print("train_features = ", train_features)
    # Convert tensor label array in dataset to regular array
    train_labels = ([torch.LongTensor.item(i[1]) for i in dataset])
    # print("train_labels = ", train_labels)
    return train_features, train_labels
```

References

- <https://stackoverflow.com/questions/49768306/pytorch-tensor-to-numpy-array>
- <https://stackoverflow.com/questions/54268029/how-to-convert-a-pytorch-tensor-into-a-numpy-array>
- <https://stackoverflow.com/questions/67711752/how-to-extract-tensors-to-numpy-arrays-or-lists-from-a-larger-pytorch-tensor>
- <https://stackoverflow.com/questions/47588682/how-to-cast-a-1-d-intensor-to-int-in-pytorch>
- <https://stackoverflow.com/questions/57727372/how-do-i-get-the-value-of-a-tensor-in-pytorch>

i[0] gets tensor feature array
i[1] gets tensor label array

Random Forest Results

- `random_forest = TorchRandomForestClassifier(50, 2000, 10)`
 - `correct = 1262/1287`
 - `sensitivity = 98.05749805749805`
- `random_forest = TorchRandomForestClassifier(200, 2000, 15)`
 - `correct = 1258/1287`
 - `sensitivity = 97.74669774669775`
- `random_forest = TorchRandomForestClassifier(200, 1500, 15)`
 - `correct = 1262/1287`
 - `sensitivity = 98.05749805749805`

Random Forest Results

- `random_forest = TorchRandomForestClassifier(15, 500, 10)`
 - `correct = 1261/1287`
 - `sensitivity = 97.97979797979798`
- `random_forest = TorchRandomForestClassifier(100, 500, 10)`
 - `correct = 1255/1287`
 - `sensitivity = 97.51359751359752`
- `random_forest = TorchRandomForestClassifier(50, 1000, 15)`
 - `correct = 1258/1287`
 - `sensitivity = 97.74669774669775`
- `random_forest = TorchRandomForestClassifier(50, 1500, 10)`
 - `correct = 1259`
 - `sensitivity = 97.82439782439782`

Decision Tree Results

- `decision_tree = TorchDecisionTreeClassifier(10)`
 - `correct = 1263`
 - `sensitivity = 98.13519813519814`
- `decision_tree = TorchDecisionTreeClassifier(20)`
 - `correct =`
 - `sensitivity =`
- `decision_tree = TorchDecisionTreeClassifier(30)`
 - `correct = 1263`
 - `sensitivity = 98.13519813519814`
- `decision_tree = TorchDecisionTreeClassifier(80)`
 - `correct = 1263`
 - `sensitivity = 98.13519813519814`

Update: 01/22/2024

Progress

- Completed the problem of Energy Band Level Hardness ratios
- Column: Flux_Band
- Printed it out. Numpy array with 8 numbers
 - `source[flux_col] = [1.318705452994784e-12, 3.918336766162156e-10, 3.155801175935835e-10, 1.0454052207231612e-10, 2.0413597895396762e-11, 6.700519479541089e-12, 1.909341825270805e-12, 2.2682772056883754e-16]`

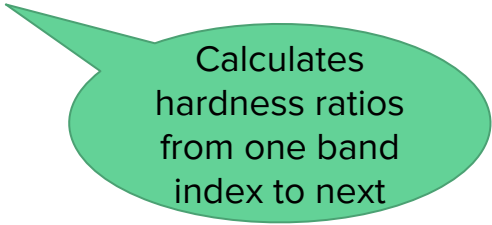
Transformation

- In FGL4Dataset.py
- Converted numpy array to list

- `flux_list = res = source[flux_col].tolist()`

- Calculated hardness ratios

```
# Calculate hardness ratios (also considered in the paper)
for index in range(1, len(flux_list)):
    unit.append((flux_list[index] - flux_list[index-1] + 0.0) /
                (flux_list[index] + flux_list[index-1] + 0.0))
xdata.append(unit)
```



Calculates
hardness ratios
from one band
index to next

Logistic Regression

- Model:
 - **Linear(in_features=20, out_features=1, bias=True)**
- Iterations = 30/100
 - Best P Threshold Value : 0.972
 - Score : 1.0586206896551724
 - Best Score : 1.9655172413793105
 - Best True Positive (AGN Successfully Identified): 280
 - AGN count : 290
 - Best AGN/True Positive/Sensitivity : 0.9655172413793104
 - Best True Negative (PSR Successfully Identified): 10
 - PSR count : 10
 - Best Pulsar/True Negative/Specificity : 1.0
 - **Accuracy of Best P Fit : 0.9666666666666667**
 - Best correct : 290
 - No of samples : 300

Neural Networks

- Model:
 - Sequential(
 - (0): Linear(in_features=20, out_features=60, bias=True)
 - (1): ReLU()
 - (2): Linear(in_features=60, out_features=1, bias=True))
- Iterations = 30
 - Best P Threshold Value : 0.976
 - Score : 1.0551724137931036
 - Best Score : 1.9586206896551723
 - Best True Positive (AGN Successfully Identified): 278
 - AGN count : 290
 - Best AGN/True Positive/Sensitivity : 0.9586206896551724
 - Best True Negative (PSR Successfully Identified): 10
 - PSR count : 10
 - Best Pulsar/True Negative/Specificity : 1.0
 - **Accuracy of Best P Fit : 0.96**
 - Best correct : 288
 - No of samples : 300

Deep Neural Networks

- Model:
 - **Sequential**
 - (0): **Linear(in_features=20, out_features=250, bias=True)**
 - (1): **ReLU()**
 - (2): **Linear(in_features=250, out_features=50, bias=True)**
 - (3): **ReLU()**
 - (4): **Linear(in_features=50, out_features=1, bias=True)**
 -)
- Results
 - Best P Threshold Value : 0.971
 - Score : 1.0586206896551724
 - Best Score : 1.9724137931034482
 - Best True Positive (AGN Successfully Identified): 282
 - AGN count : 290
 - Best AGN/True Positive/Sensitivity : 0.9724137931034482
 - Best True Negative (PSR Successfully Identified): 10
 - PSR count : 10
 - Best Pulsar/True Negative/Specificity : 1.0
 - **Accuracy of Best P Fit : 0.9733333333333334**
 - Best correct : 292
 - No of samples : 300

Deep Neural Networks

- DNN
- Model:
 - Sequential(
 - (0): Linear(in_features=20, out_features=250, bias=True)
 - (1): ReLU()
 - (2): Linear(in_features=250, out_features=40, bias=True)
 - (3): ReLU()
 - (4): Linear(in_features=40, out_features=1, bias=True))
 - Best P Threshold Value : 0.967
 - Score : 1.0517241379310345
 - Best Score : 1.9689655172413794
 - Best True Positive (AGN Successfully Identified): 281
 - AGN count : 290
 - Best AGN/True Positive/Sensitivity : 0.9689655172413794
 - Best True Negative (PSR Successfully Identified): 10
 - PSR count : 10
 - Best Pulsar/True Negative/Specificity : 1.0
 - **Accuracy of Best P Fit : 0.97**
 - Best correct : 291
 - No of samples : 300

Random Forests

- Model:
 - **# Trees: 200**
 - **# Samples: 2500**
 - **Depth: 15**
- Results
 - Correct = 1258
 - Total = 1286
 - Sensitivity = 97.82439782439782

Random Forests

- Model:
 - **# Trees: 200**
 - **# Samples: 1500**
 - **Depth: 15**
- Results
 - Correct = 1258
 - Total = 1286
 - Sensitivity = 97.74669774669775

Random Forests

- Model:
 - **# Trees: 200**
 - **# Samples: 500**
 - **Depth: 15**
- Results
 - Correct = 1254
 - Total = 1286
 - Sensitivity = 97.43589743589743

Decision Trees

- Decision Trees
- Model:
 - **Depth: 20**
 - Correct = 1259
 - Total = 1286
 - Sensitivity = 97.82439782439782

Next Steps

- Could you kindly clarify what needs to be done with the following tasks
 - associated/unassociated sources
 - AGN types

Update: 02/09/2024

Progress

- Simplified Logistic Regression & Neural Networks Python code to train and then test on the entire 30% test dataset
- This resolves the earlier problem where the test was showing only PSR count = 10 & AGN count = 290
- New scripts:
 - LR: 4FGLReproductionBasicLRnew.py
 - NN: 4FGLNNnew.py
 - DNN: 4FGLDNNnew.py
 - Only difference in the above scripts is the model trained and tested

Training

```
torch.set_default_tensor_type(torch.DoubleTensor)
torch.manual_seed(82)
LogRegModel = LogisticRegression(train_dataset)
# Stochastic Gradient Descent. I could use Adams but Adams is worse than regular SGD unless you
optimizer = torch.optim.SGD(LogRegModel.parameters(), lr=0.0001)
print("optimizer = ", optimizer.__class__)
# Binary Cross Entropy Loss which is the loss method that would most likely be used in this scenario
criterion = nn.BCELoss()
tot_iter = 100
print("total iterations = ", tot_iter)
for iteration in range(0, tot_iter):
    dev_inputs = dev_labels = []
    print("Iteration # ", iteration)
    total_samples = len(train_dataset)
    # KrishivB: Put run_iteration code in utils.py, imported it, and call here
    for epoch in range(30):
        (dev_inputs, dev_labels) = run_iteration(LogRegModel, iteration, train_dataset, total_samples, criterion,
                                                optimizer, dev_inputs, dev_labels, append_dev_input_label: True)
```

Define model once

Iterations for loop

Train model in iterations

Testing

Enumerate over
test dataset

Predict

Get actual
value

Check if predicted
matches with actual

```
correct = correct_agm = correct_psr = total_agm = total_psr = 0
for i, (inputs, labels) in enumerate(test_dataset):
    y_predicted = LogRegModel(inputs) # Insert/fit Model for prediction here
    predicted_item = y_predicted.data.item()
    print(" i, y_predicted, y_predicted.data, predicted_item = ", i, y_predicted, y_predicted.data, predicted_item)
    predicted_val = 0 if (math.isnan(predicted_item)) else round(predicted_item)
    label_val = labels.item()
    total_agm += 1 if label_val == 1 else 0
    total_psr += 1 if label_val == 0 else 0
    match = (predicted_val == label_val)
    if match:
        correct += 1
        correct_agm += 1 if label_val == 1 else 0
        correct_psr += 1 if label_val == 0 else 0
    print("i, predicted_item, predicted_val, label_val, correct, match = ",
          i, predicted_item, predicted_val, label_val, correct, match)
```

Results

Logistic Regression

- # iterations = 50
- Results
 - Correct = 1261
 - correct_agn, correct_psr = 1241 20
 - total, total_agn, total_psr = 1287 1244 43
 - **total sensitivity = 0.9797979797979798**
 - agn sensitivity = 0.997588424437299
 - psr sensitivity = 0.46511627906976744

- # iterations = 750
- Results
 - Correct = 1261
 - correct_agn, correct_psr = 1239 22
 - total, total_agn, total_psr = 1287 1244 43
 - **total sensitivity = 0.9797979797979798**
 - agn sensitivity = 0.9959807073954984
 - psr sensitivity = 0.5116279069767442

- # iterations = 100, 150, 200
- Results
 - Correct = 1262
 - correct_agn, correct_psr = 1240 22
 - total, total_agn, total_psr = 1287 1244 43
 - **total sensitivity = 0.9805749805749806**
 - agn sensitivity = 0.9967845659163987
 - psr sensitivity = 0.5116279069767442

- # iterations = 1000, 1250
- Results
 - Correct = 1261
 - correct_agn, correct_psr = 1238 23
 - total, total_agn, total_psr = 1287 1244 43
 - **total sensitivity = 0.9797979797979798**
 - agn sensitivity = 0.9951768488745981
 - psr sensitivity = 0.5348837209302325

Neural Network

- # iterations = 10
- Results
 - Correct = 1261
 - correct_agm, correct_psr = 1241 20
 - total, total_agm, total_psr = 1287 1244 43
 - total sensitivity = 0.9797979797979798
 - agm sensitivity = 0.997588424437299
 - psr sensitivity = 0.46511627906976744

- # iterations = 50
- Results
 - Correct = 1265
 - correct_agm, correct_psr = 1240 25
 - total, total_agm, total_psr = 1287 1244 43
 - total sensitivity = 0.9829059829059829
 - agm sensitivity = 0.9967845659163987
 - psr sensitivity = 0.5813953488372093

- # iterations = 100
- Results
 - Correct = 1264
 - correct_agm, correct_psr = 1239 25
 - total, total_agm, total_psr = 1287 1244 43
 - total sensitivity = 0.9821289821289821
 - agm sensitivity = 0.9959807073954984
 - psr sensitivity = 0.5813953488372093

- # iterations = 150
- Results
 - Correct = 1265
 - correct_agm, correct_psr = 1237 28
 - total, total_agm, total_psr = 1287 1244 43
 - total sensitivity = 0.9829059829059829
 - agm sensitivity = 0.9943729903536977
 - psr sensitivity = 0.6511627906976745

Deep Neural Network

- # iterations = 10
- Results
 - Correct = 1260
 - correct_agn, correct_psr = 1241 19
 - total, total_agn, total_psr = 1287 1244 43
 - total sensitivity = 0.9790209790209791
 - agn sensitivity = 0.997588424437299
 - psr sensitivity = 0.4418604651162791

- # iterations = 25
- Results
 - Correct = 1266
 - correct_agn, correct_psr = 1240 26
 - total, total_agn, total_psr = 1287 1244 43
 - total sensitivity = 0.9836829836829837
 - agn sensitivity = 0.9967845659163987
 - psr sensitivity = 0.6046511627906976

- # iterations = 100
- Results
 - Correct = 1267
 - correct_agn, correct_psr = 1239 28
 - total, total_agn, total_psr = 1287 1244 43
 - total sensitivity = 0.9844599844599845
 - agn sensitivity = 0.9959807073954984
 - psr sensitivity = 0.6511627906976745

- # iterations = 150
- Results
 - Correct = 1267
 - correct_agn, correct_psr = 1236 29
 - total, total_agn, total_psr = 1287 1244 43
 - total sensitivity = 0.9829059829059829
 - agn sensitivity = 0.9935691318327974
 - psr sensitivity = 0.6744186046511628

Observations

- I found overall AGN sensitivity to be > 0.995 throughout but PSR sensitivity to be around 0.5.
- With the NN & DNN model, PSR sensitivity started going up with more training (as # iterations increased).
- In the case of NN with 150 iterations (each iteration having 30 epochs), PSR sensitivity increased to 0.65.
- In the case of DNN, with 25 iterations, PSR sensitivity increased to 0.60 and with 100 iterations, increased to 0.65, and increased to 0.67 with 150 iterations.
- So we may need to find what is the problem with PSR?
 - Is it related to the choice of model, features, or training?

Update: 02/19/2024

Progress

- Fix the problem of low Pulsar count
- Added “msp” in wantedtypes list



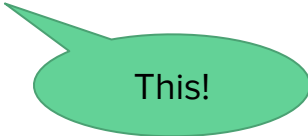
This!

```
|# KrishivB: Added AGN in caps  
|wantedtypes = ["PSR", "psr", "YNG", "yng", "MSP", "msp", "FSRQ", "fsrq", "BLL", "bll", "BCU", "bcu", "RDG", "rdg",  
|              "NLSY1", "nlsy1", "AGN", "agn", "ssrq", "sey"]
```

Progress

- Added “MSP”, “msp” to the **source[wanted_type_col]** check in FGL4Dataset.py
- It was missing. Only “PSR”, “psr” were present.

```
# Calculate hardness ratios (also considered in the paper)
for index in range(1, len(flux_list)):
    unit.append((flux_list[index] - flux_list[index-1] + 0.0) /
                (flux_list[index] + flux_list[index-1] + 0.0))
xdata.append(unit)
if source[wanted_type_col] in ("PSR", "psr", "MSP", "msp"):
    ydata.append([0]) # 0 is for pulsar
else:
    ydata.append([1]) # 1 is for AGN
```



This!

Logistic Regression

- # iterations = 25
- Results
 - Correct = 1260
 - correct_agn, correct_psr = 1172 88
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9699769053117783**
 - agn sensitivity = 0.9890295358649789
 - psr sensitivity = 0.7719298245614035

- # iterations = 200, 500
- Results
 - Correct = 1264
 - correct_agn, correct_psr = 1177 87
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9730561970746728**
 - agn sensitivity = 0.9932489451476794
 - psr sensitivity = 0.7631578947368421

- # iterations = 50, 100, 150
- Results
 - Correct = 1264
 - correct_agn, correct_psr = 1177 87
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9730561970746728**
 - agn sensitivity = 0.9932489451476794
 - psr sensitivity = 0.7631578947368421

- # iterations = 300
- Results
 - Correct = 1265
 - correct_agn, correct_psr = 1178 87
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9738260200153964**
 - agn sensitivity = 0.9940928270042194
 - psr sensitivity = 0.7631578947368421

Neural Network

- # iterations = 25
- Results
 - Correct = 1261
 - correct_agn, correct_psr = 1173 88
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.970746728252502**
 - agn sensitivity = 0.9898734177215189
 - psr sensitivity = 0.7719298245614035

- # iterations = 200
- Results
 - Correct = 1261
 - correct_agn, correct_psr = 1178 83
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.970746728252502**
 - agn sensitivity = 0.9940928270042194
 - psr sensitivity = 0.7280701754385965

- # iterations = 50, 100
- Results
 - Correct = 1257
 - correct_agn, correct_psr = 1176 81
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9676674364896074**
 - agn sensitivity = 0.9924050632911392
 - psr sensitivity = 0.7105263157894737

- # iterations = 300
- Results
 - Correct = 1262
 - correct_agn, correct_psr = 1180 82
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9715165511932256**
 - agn sensitivity = 0.9957805907172996
 - psr sensitivity = 0.7192982456140351

Deep Neural Network

- # iterations = 25
- Results
 - Correct = 1260
 - correct_agn, correct_psr = 1172 88
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9699769053117783**
 - agn sensitivity = 0.9890295358649789
 - psr sensitivity = 0.7719298245614035

- # iterations = 200
- Results
 - Correct = 1260
 - correct_agn, correct_psr = 1175 85
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9699769053117783**
 - agn sensitivity = 0.9915611814345991
 - psr sensitivity = 0.7456140350877193

- # iterations = 100
- Results
 - Correct = 1258
 - correct_agn, correct_psr = 1176 82
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.968437259430331**
 - agn sensitivity = 0.9924050632911392
 - psr sensitivity = 0.7192982456140351

- # iterations = 300
- Results
 - Correct = 1260
 - correct_agn, correct_psr = 1173 87
 - total, total_agn, total_psr = 1299 1185 114
 - **total sensitivity = 0.9699769053117783**
 - agn sensitivity = 0.9898734177215189
 - psr sensitivity = 0.7631578947368421

Random Forest

- **Model**

- # Trees: 200
- # Samples: 500
- Depth: 15

- **Results**

- Correct = 1245
- sensitivity = 95.84295612009238

- **Model**

- # Trees: 300
- # Samples: 1500
- Depth: 15

- **Results**

- Correct = 1251
- sensitivity = 96.30484988452656

- **Model**

- # Trees: 200
- # Samples: 1000
- Depth: 15

- **Results**

- Correct = 1246
- sensitivity = 95.91993841416475

Update: 02/26/2024

Progress

- Classified unassociated sources as PSR or AGN
 - Initially, trained and tested models with regular dataset
 - Then, read dataset with field Class1 == null
 - Used trained model to predict whether the unassociated sources are PSRs or AGNs
- Also, plotted measure of outlyingness of the unassociated sources from the class.

Outlyingness Measure

- Predictions ranged from [0.0, 1.0]
- AGNs
 - Predictions were in range [0.5, 1.0]
 - The more near it was to 1.0, the closer it was to the class
 - The farther away it was from 1.0 (or closer to 0.5), the more outlying it was from the class
- PSRs
 - Predictions were in range [0.0, 0.5]
 - The more near it was 0.0, the closer it was to the class
 - The farther away it was from 0.0 (or closer to 0.5), the more outlying it was from the class

Code change

- New files

- Logistic Regression
 - 4FGL_LR_cls1_null.py
- Neural Networks
 - 4FGL_NN_cls1_null.py
 - 4FGL_DNN_cls1_null.py
- Random Forests
 - 4FGL_RF_cls1_null.py
- Utils
 - FGL4Dataset_cls1_null.py



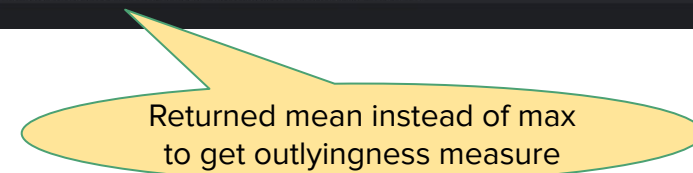
Read dataset with field Class1 == null

- Updated files

- Random Forests
 - random_forest.py



```
11 11 11
predictions = []
for tree, index_features in zip(self.trees, self.trees_features):
    sampled_vector = torch.index_select(vector, dim=0, index=index_features)
    predictions.append(tree.predict(sampled_vector))
print("predictions=", predictions)
print("sum(predictions)={}, len(self.trees)={}, sum(predictions)/len(self.trees)={}",
      sum(predictions), len(self.trees), sum(predictions)/len(self.trees))
return sum(predictions)/len(self.trees)
# return max(set(predictions), key=predictions.count)
```



Returned mean instead of max
to get outlyingness measure

Code change

- FGL4Dataset_cls1_null.py

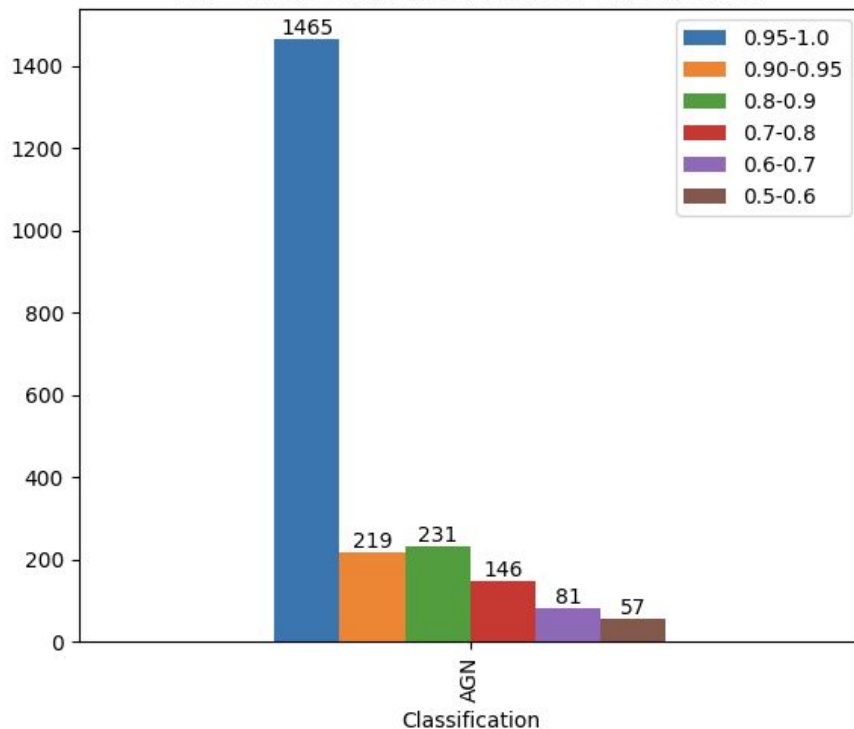
```
class FGL4Dataset_Cls1_Null(Dataset):
    # Krishiv Bhatia
    def __init__(self, wanted_type_col, flux_col, wanted_types, wanted_feature_names, poi
        # Create x/input data
        xdata = []
        ydata = []
        # Select sources that are a part of the wanted types
        for source in point_source_catalogue.data:
            # for feature_names in wanted_feature_names:
            #     print("{0} = {1}".format(feature_names, source[feature_names]))
            # print("(type(source[flux_col]) = ", type(source[flux_col]))
            flux_list = res = source[flux_col].tolist()
            # print("source[flux_col] = {0}".format(flux_list))
            print("source[wanted_type_col] = ", source[wanted_type_col])
            if source[wanted_type_col] == '':
                unit = []
                # Append wanted params. Applied a log transformation to paras with highly
                unit.append(float(source[wanted_feature_names[0]]))
                unit.append(float(source[wanted_feature_names[1]]))
                unit.append(float(source[wanted_feature_names[2]]))
                unit.append(float(source[wanted_feature_names[3]]))
```

Read dataset with field Class1 == null

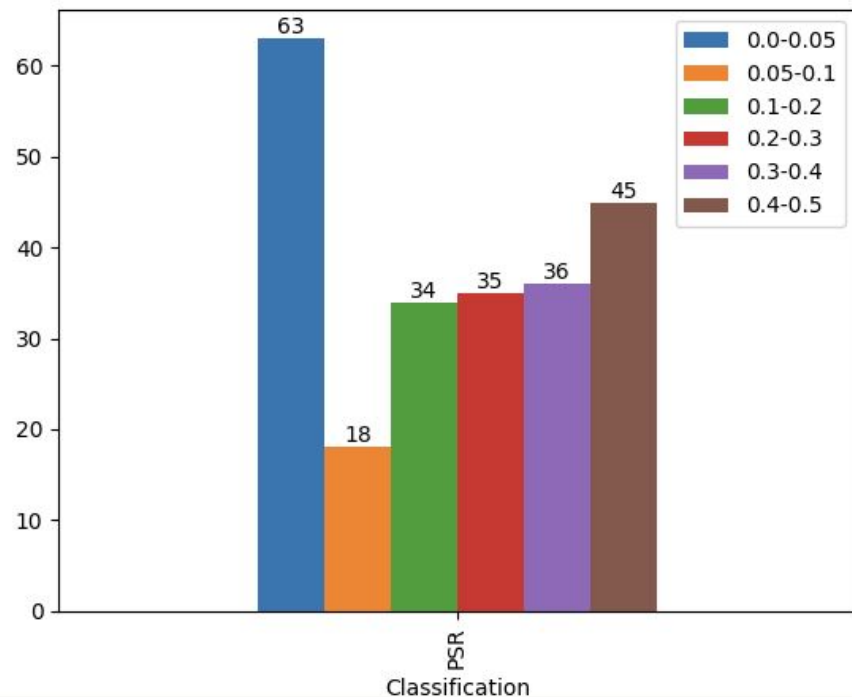
Logistic Regression

- # iterations = 50 Total: 2430 unassociated sources

2199 unassociated sources AGN classification



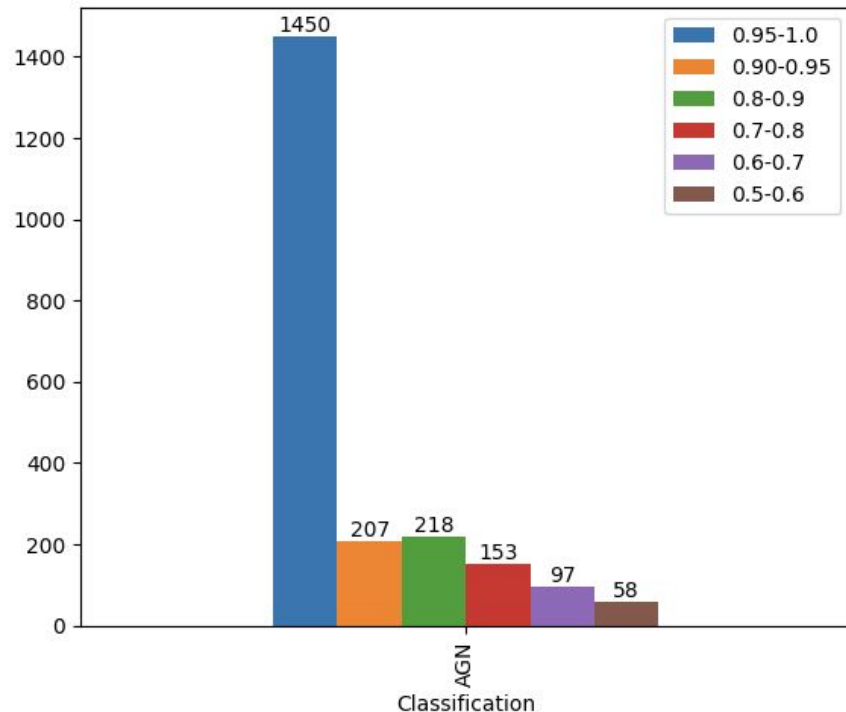
231 unassociated sources PSR classification



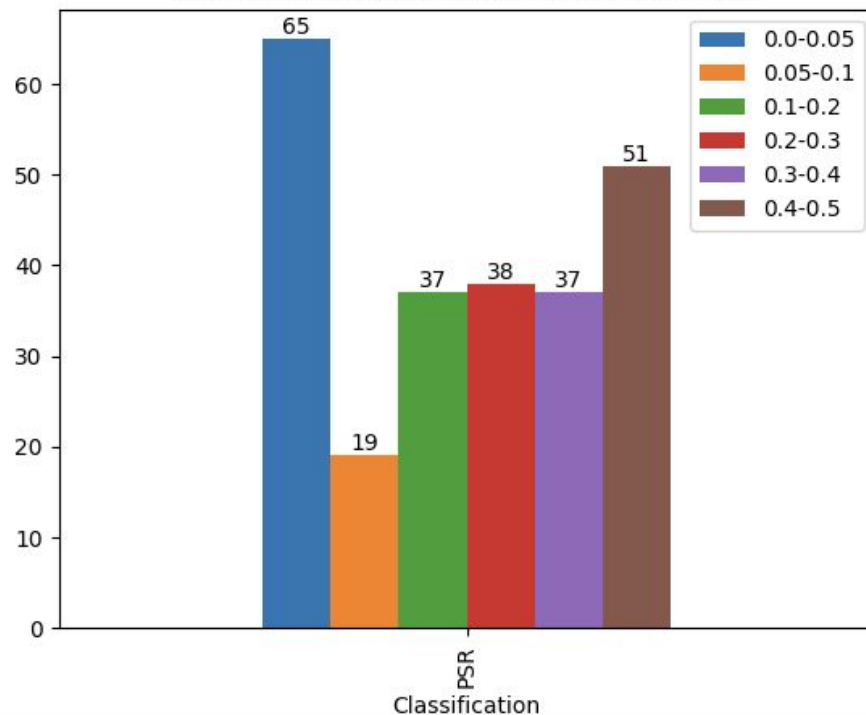
Logistic Regression

- # iterations = 100 Total: 2430 unassociated sources

2183 unassociated sources AGN classification



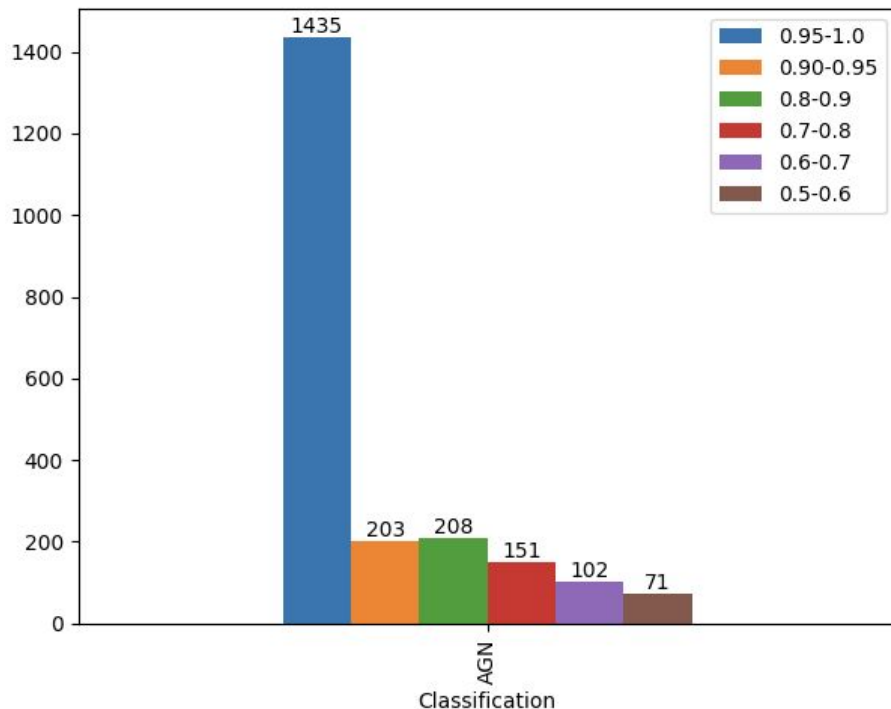
247 unassociated sources PSR classification



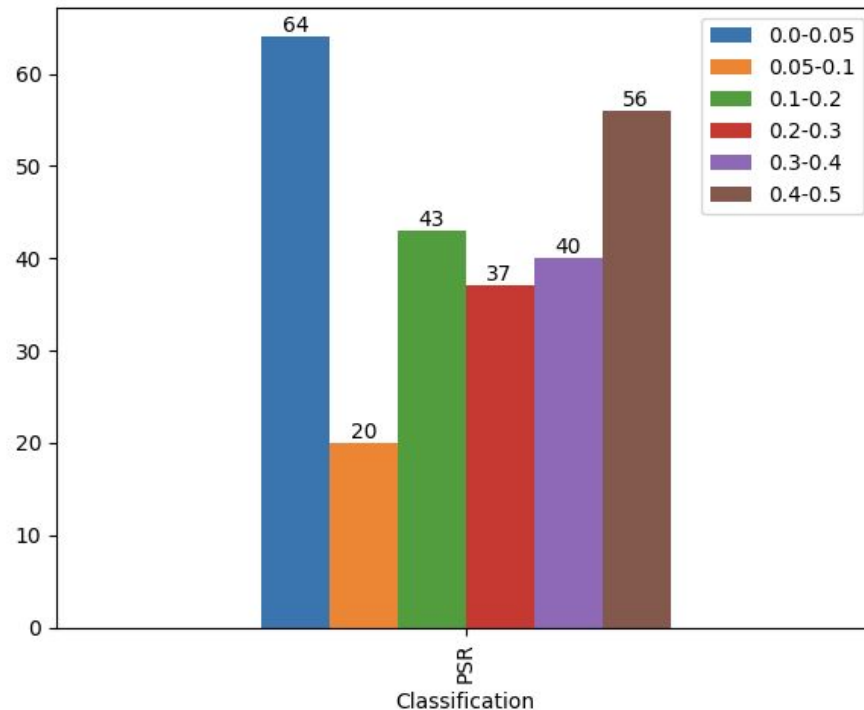
Logistic Regression

- # iterations = 200 Total: 2430 unassociated sources

2170 unassociated sources AGN classification



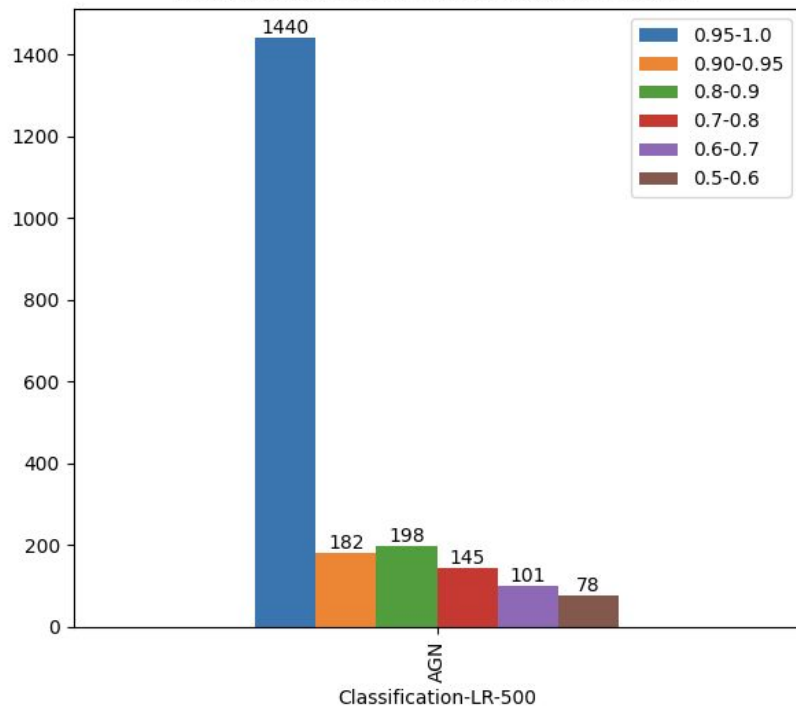
260 unassociated sources PSR classification



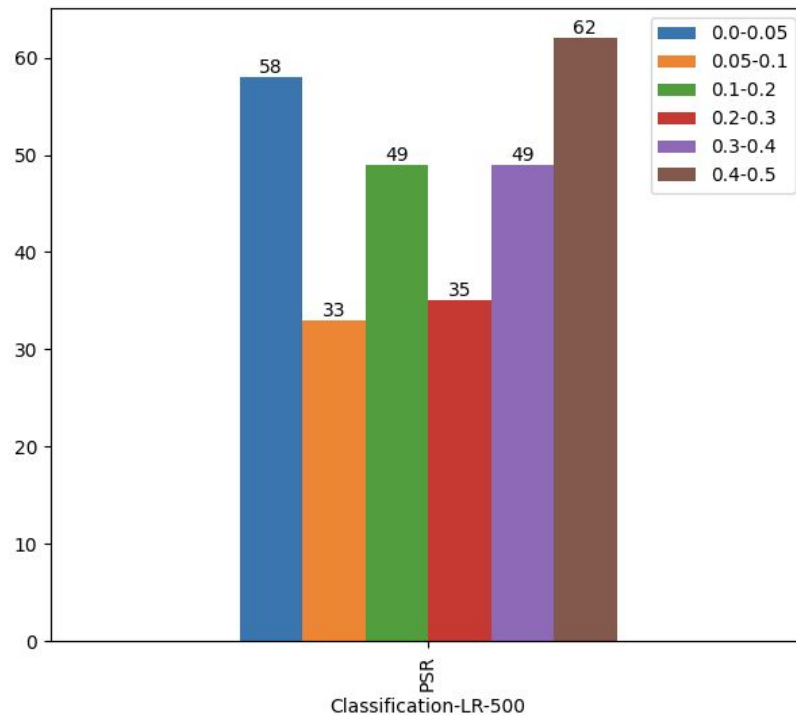
Logistic Regression

- # iterations = 500 Total: 2430 unassociated sources

2144 unassociated sources AGN classification

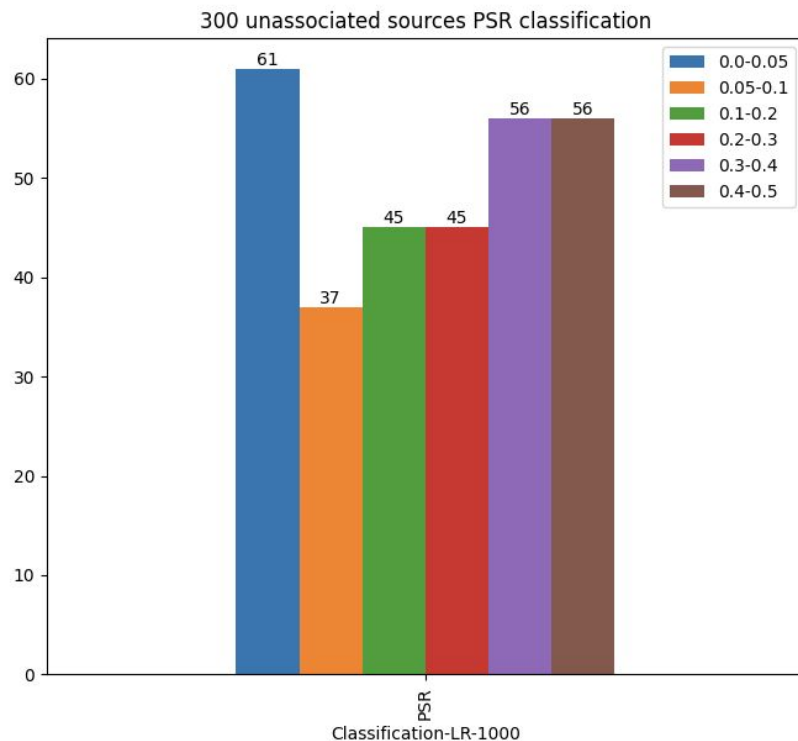
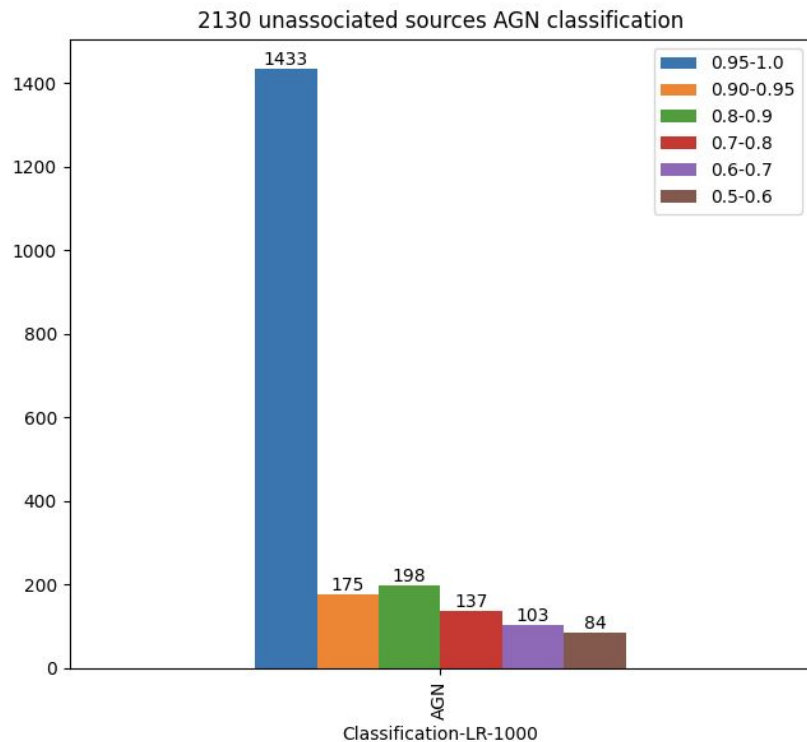


286 unassociated sources PSR classification



Logistic Regression

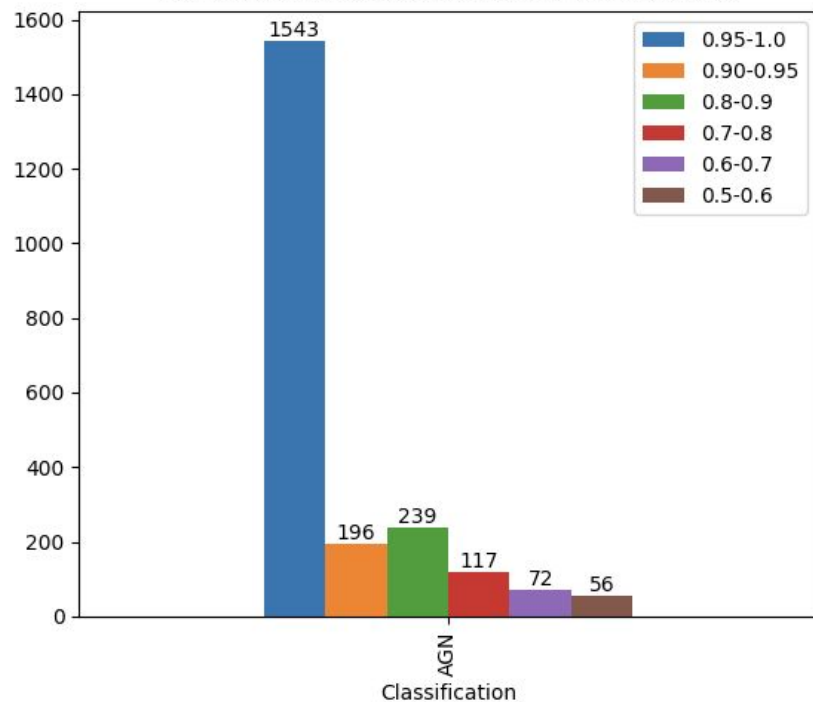
- # iterations = 1000 Total: 2430 unassociated sources



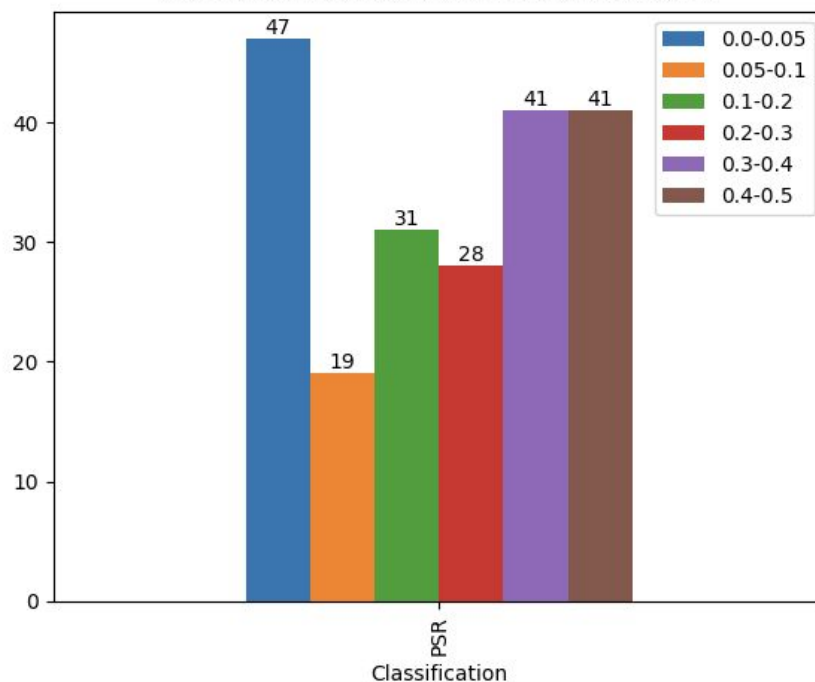
Neural Networks

- # iterations = 50 Total: 2430 unassociated sources

2223 unassociated sources AGN classification



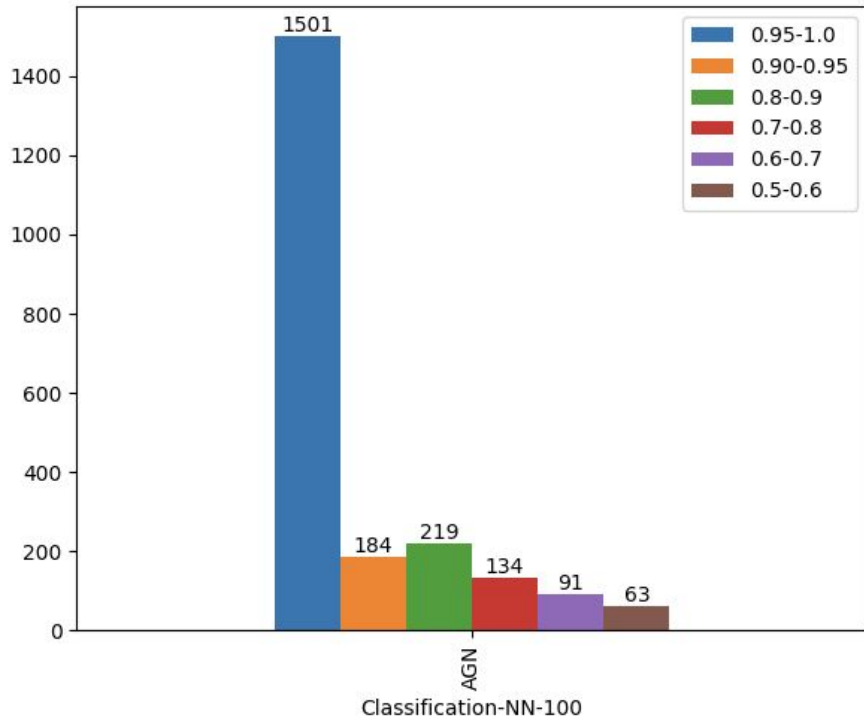
207 unassociated sources PSR classification



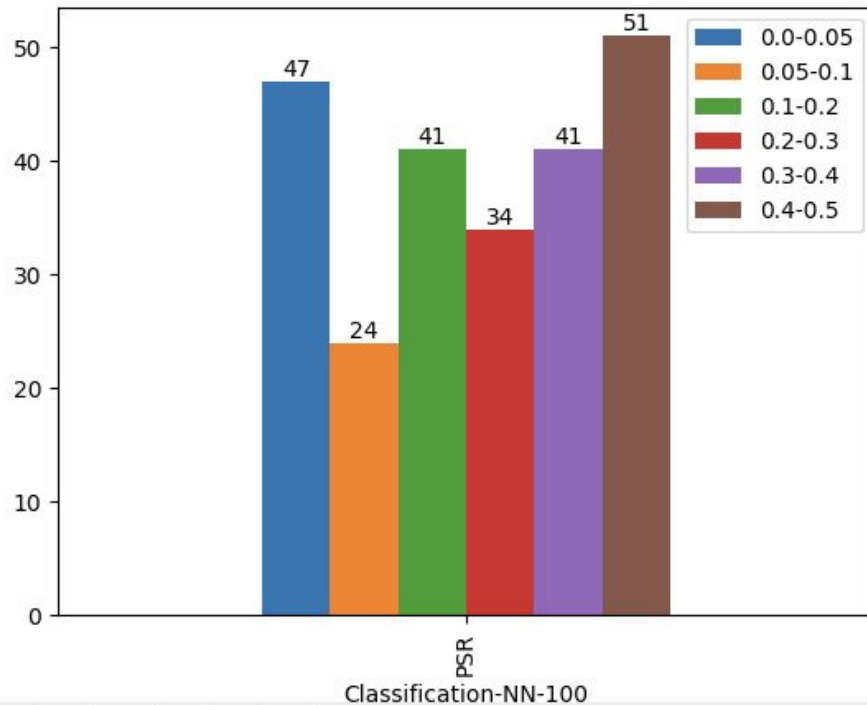
Neural Networks

- # iterations = 100 Total: 2430 unassociated sources

2192 unassociated sources AGN classification



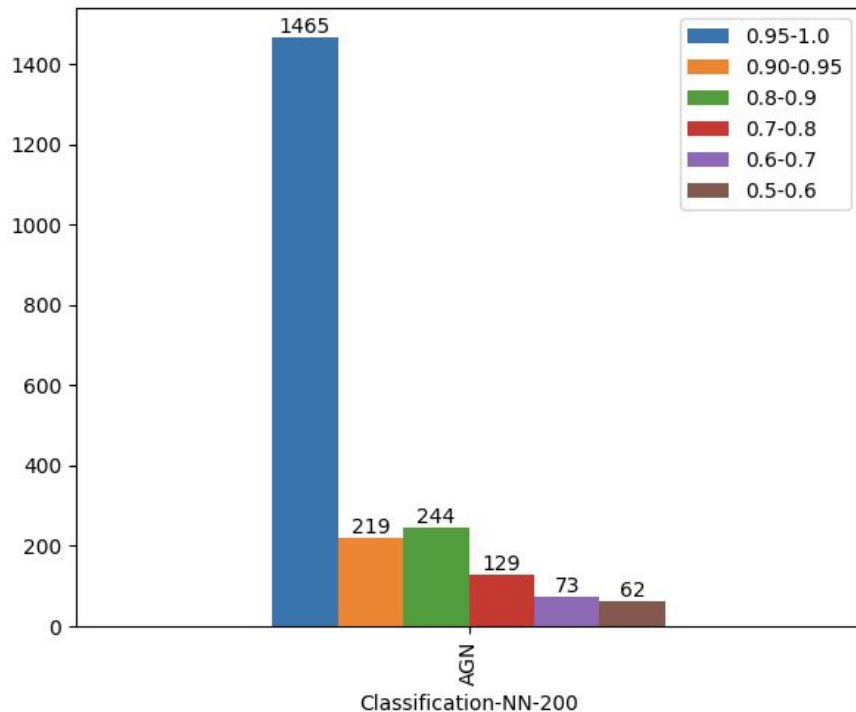
238 unassociated sources PSR classification



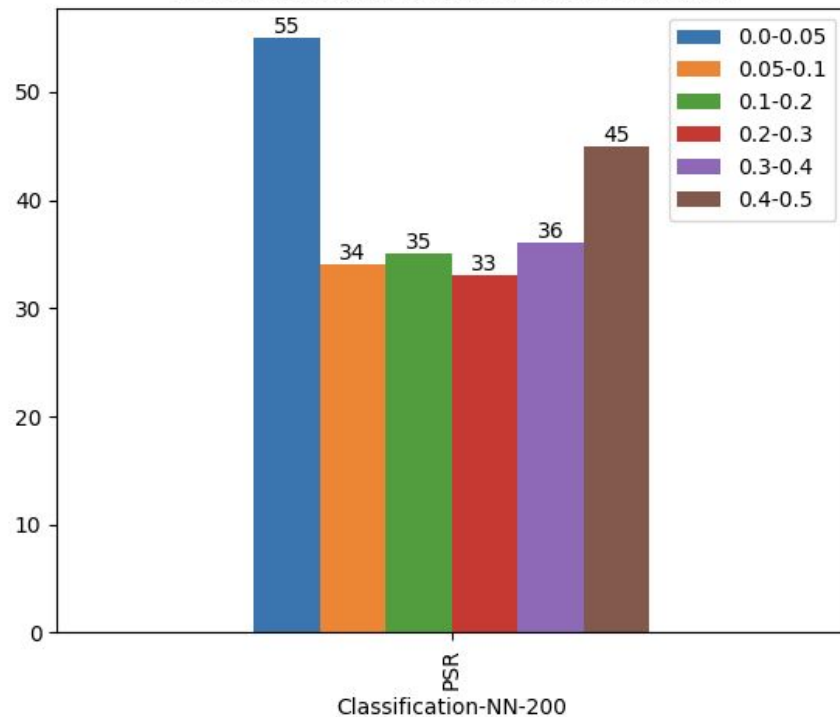
Neural Networks

- # iterations = 200 Total: 2430 unassociated sources

2192 unassociated sources AGN classification



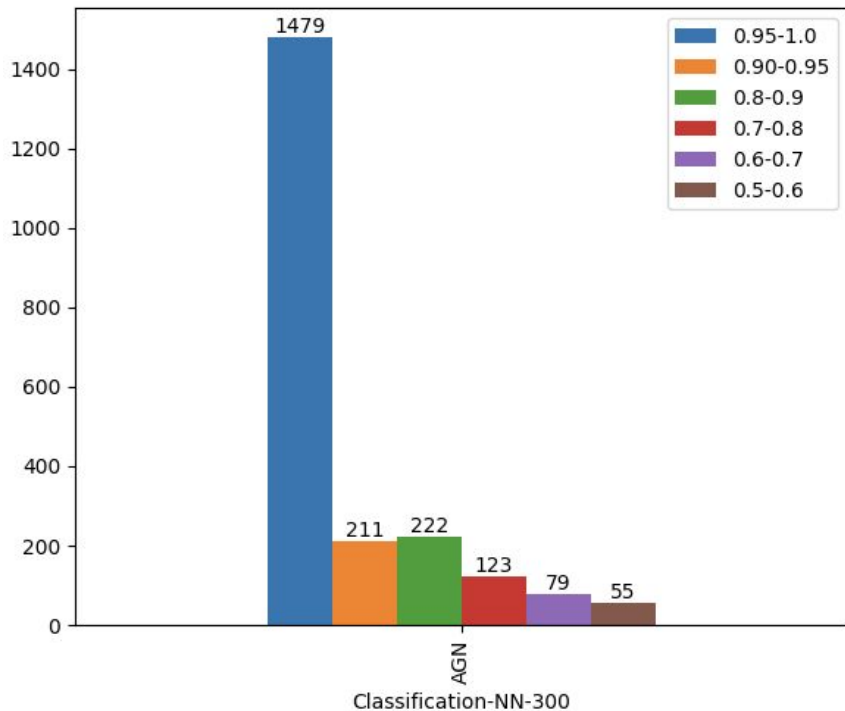
238 unassociated sources PSR classification



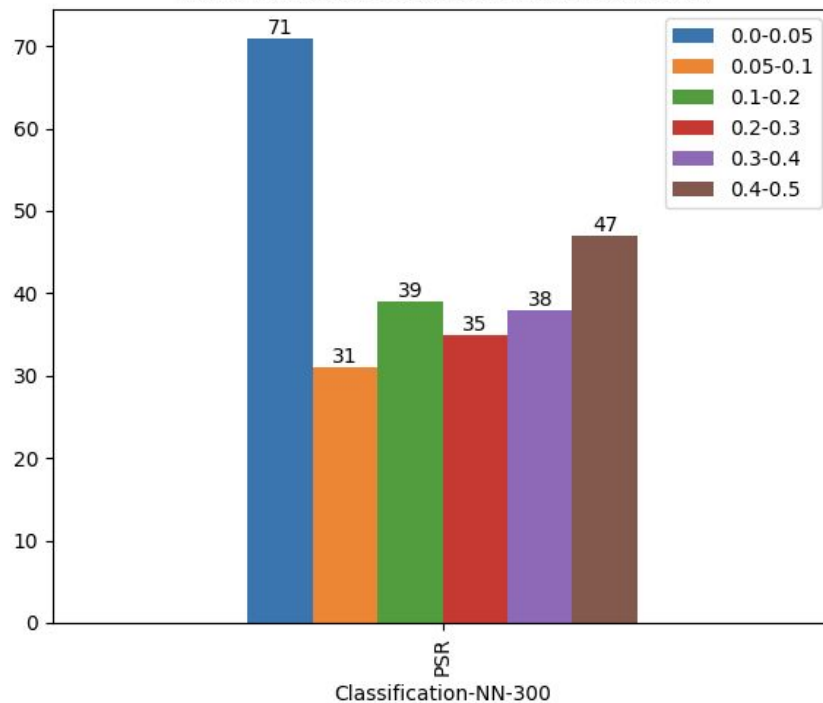
Neural Networks

- # iterations = 300 Total: 2430 unassociated sources

2169 unassociated sources AGN classification



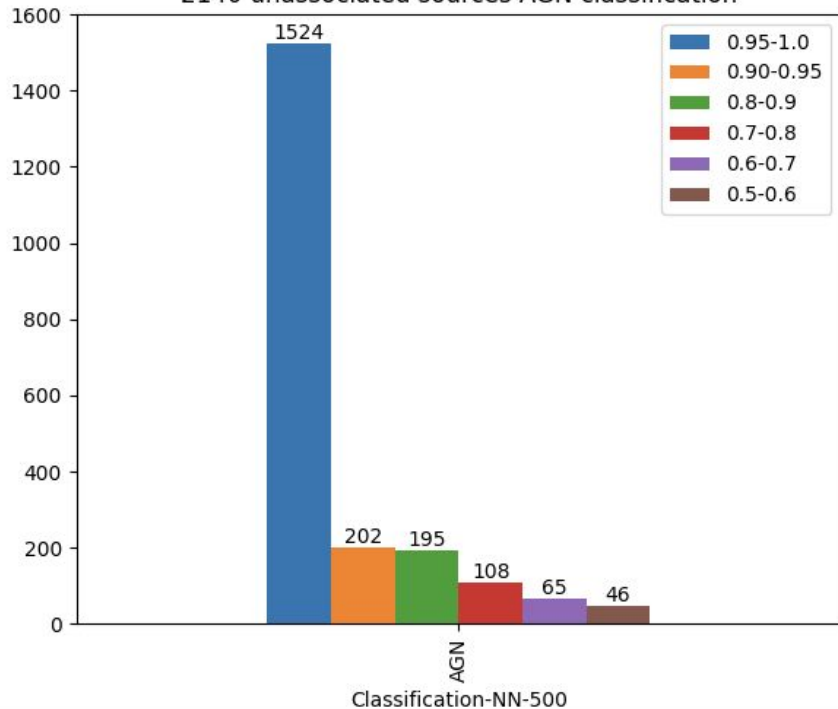
261 unassociated sources PSR classification



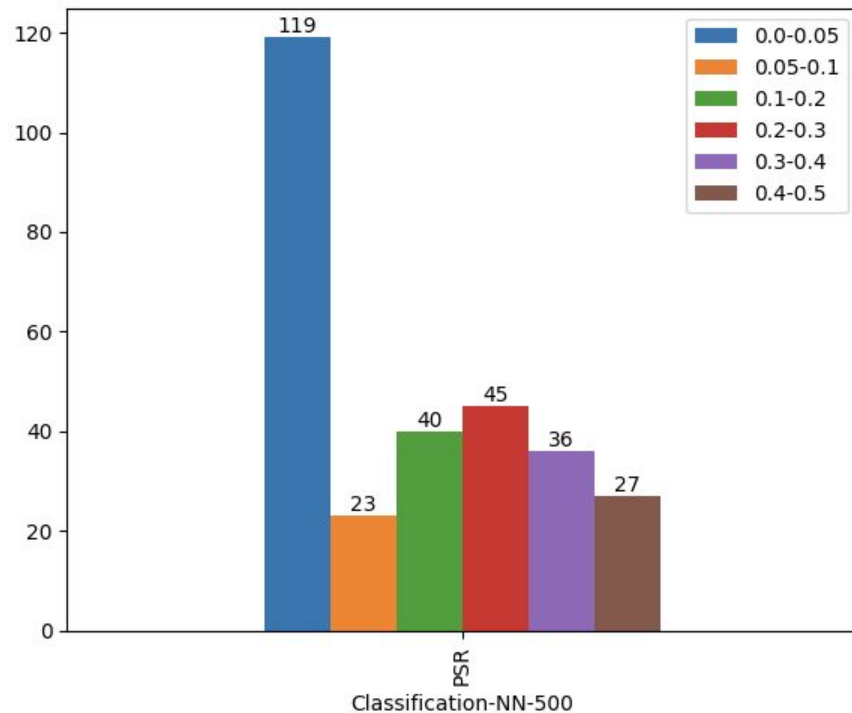
Neural Networks

- # iterations = 500 Total: 2430 unassociated sources

2140 unassociated sources AGN classification



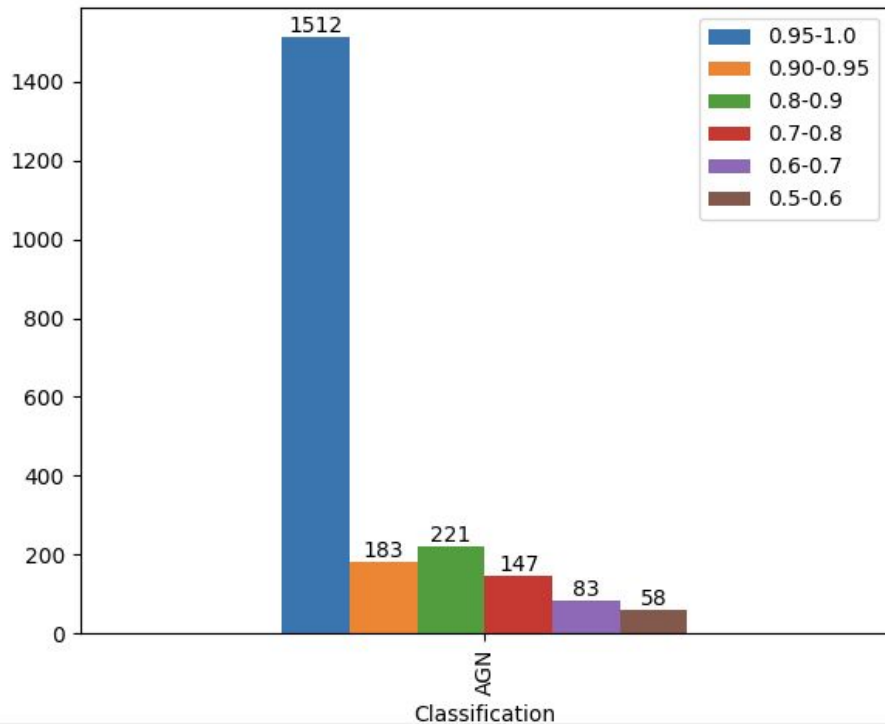
290 unassociated sources PSR classification



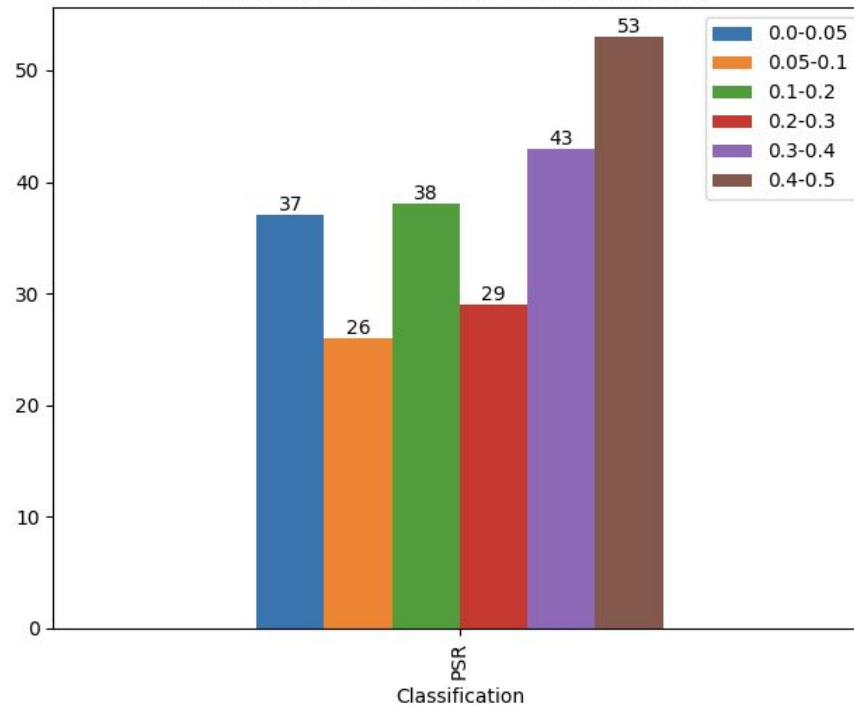
Deep Neural Networks

- # iterations = 50 Total: 2430 unassociated sources

2204 unassociated sources AGN classification



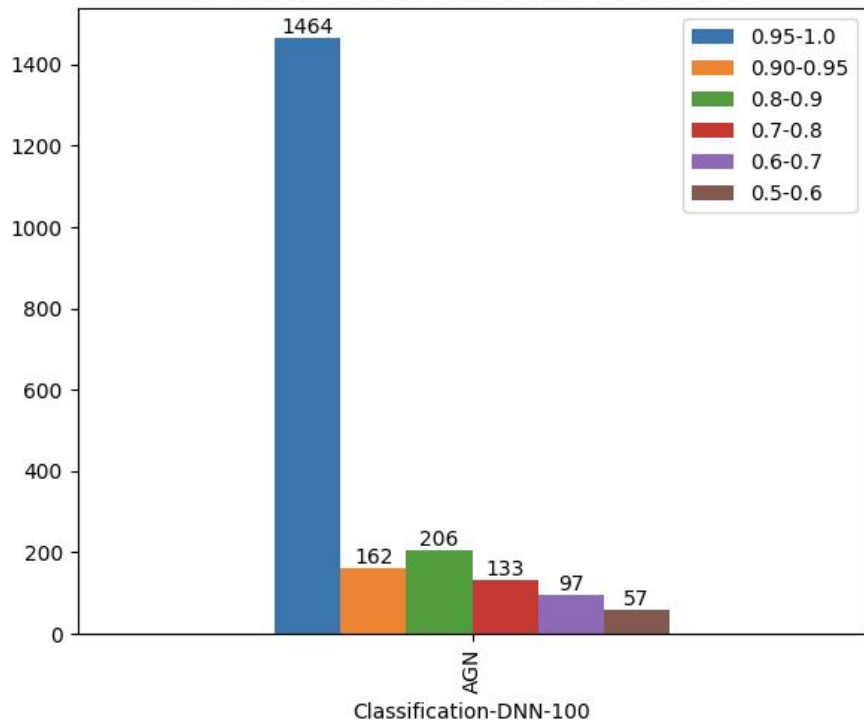
226 unassociated sources PSR classification



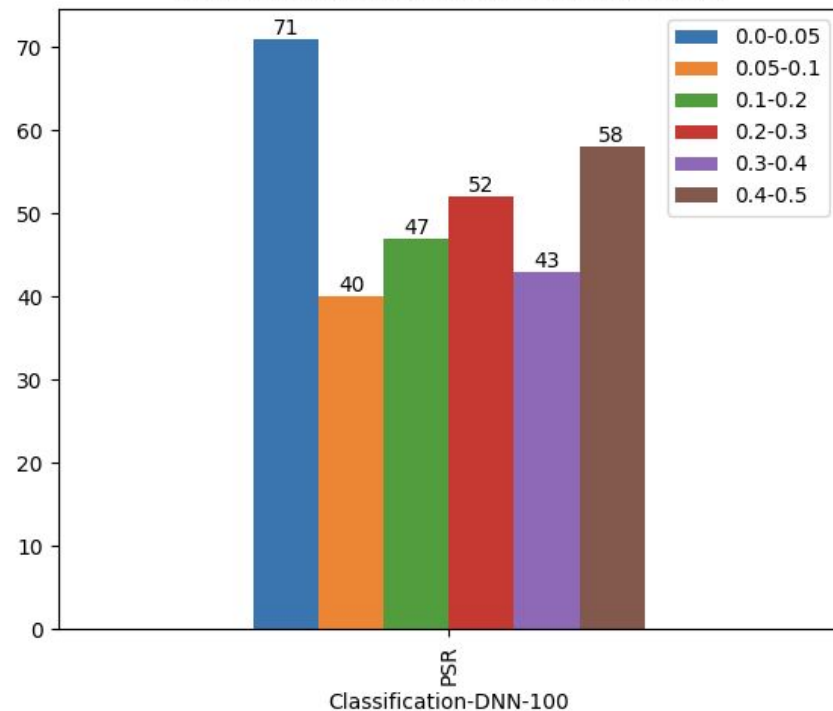
Deep Neural Networks

- # iterations = 100 Total: 2430 unassociated sources

2119 unassociated sources AGN classification



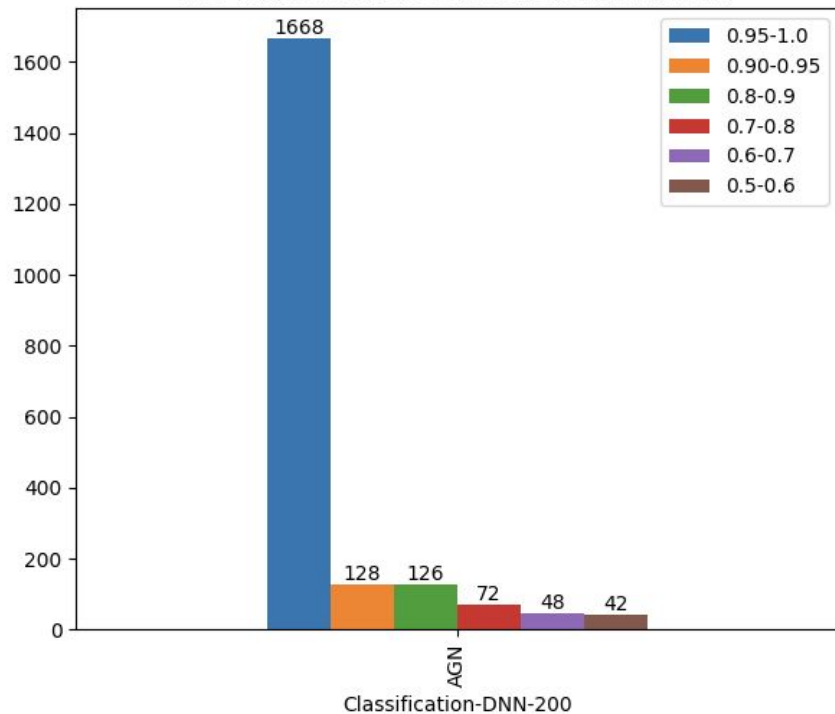
311 unassociated sources PSR classification



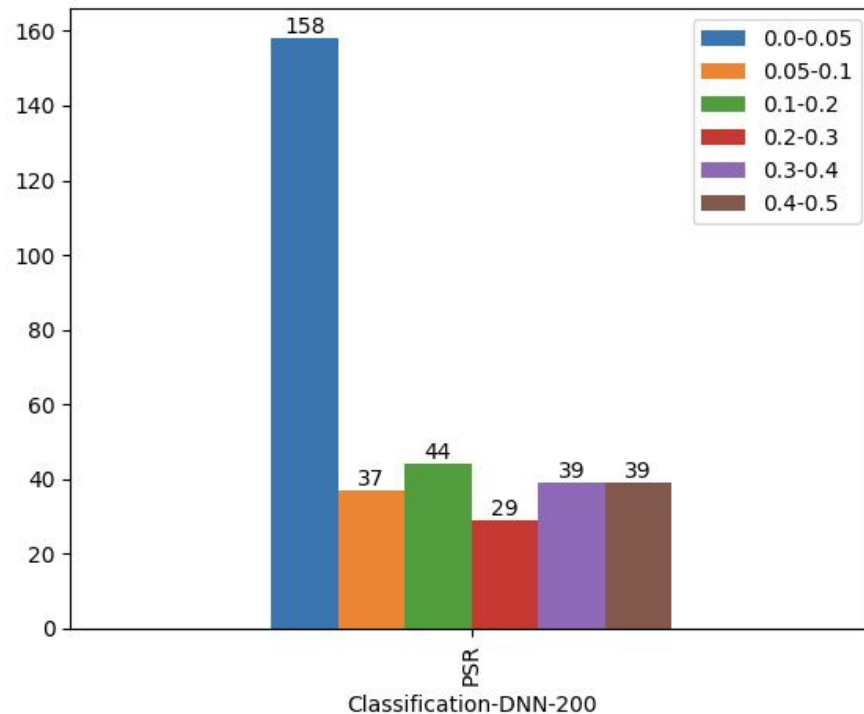
Deep Neural Networks

- # iterations = 200 Total: 2430 unassociated sources

2084 unassociated sources AGN classification

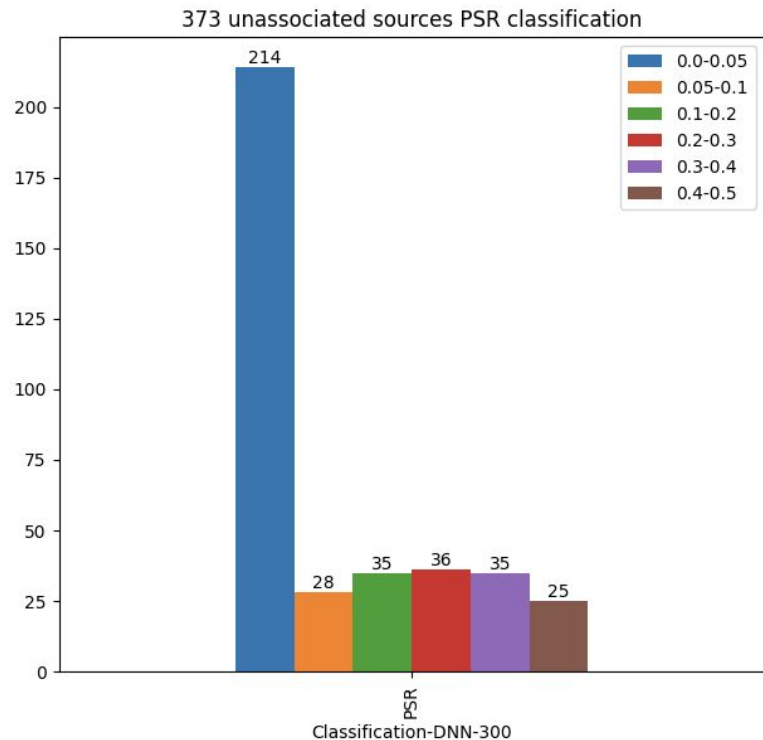
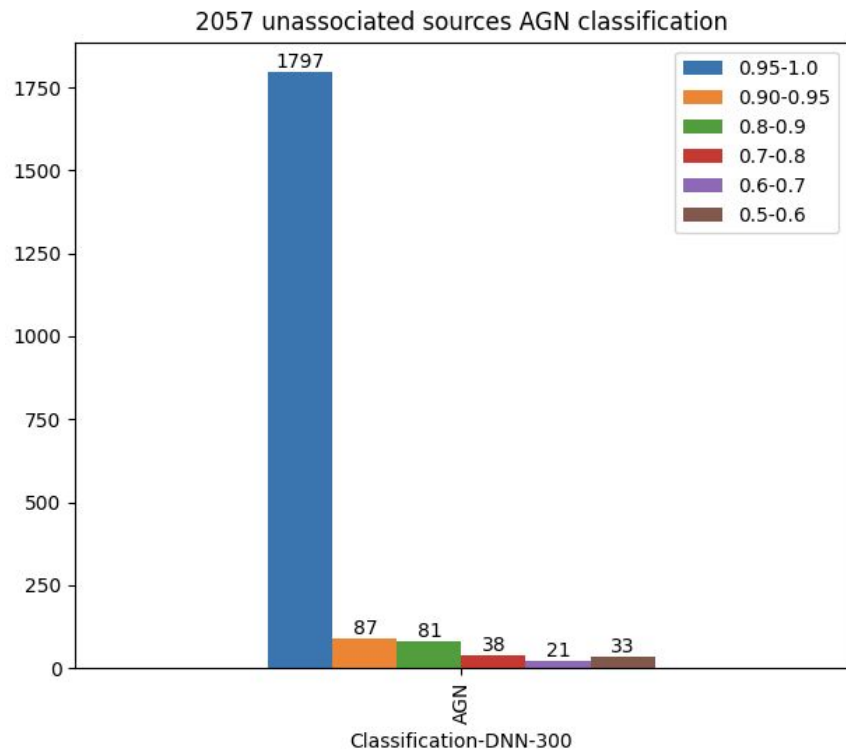


346 unassociated sources PSR classification



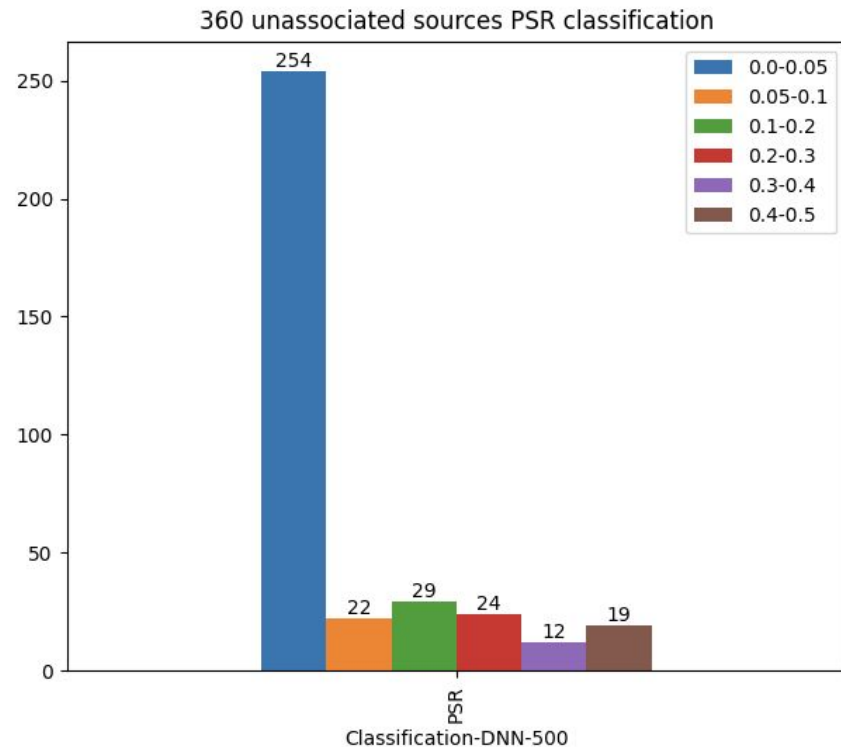
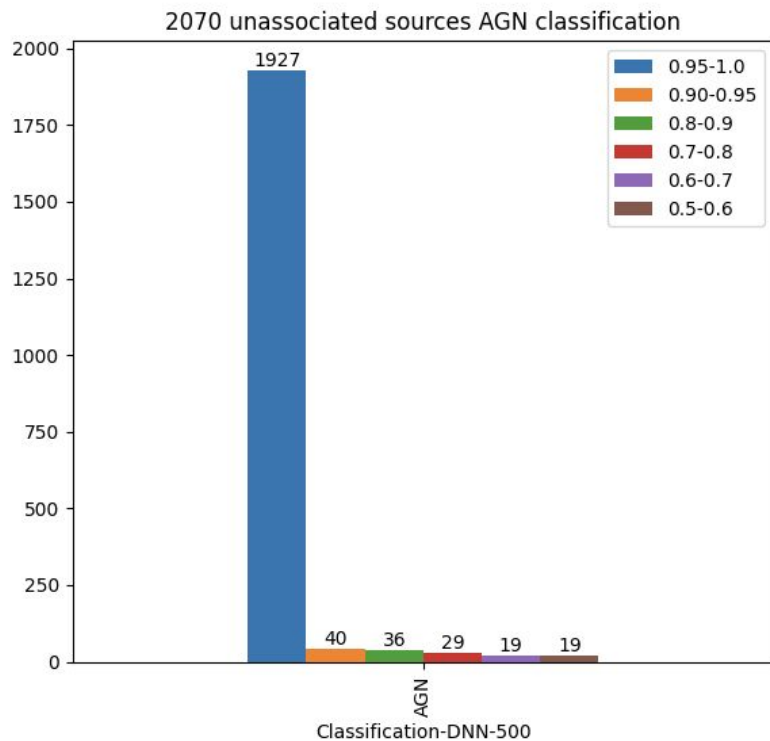
Deep Neural Networks

- # iterations = 300 Total: 2430 unassociated sources



Deep Neural Networks

- # iterations = 500 Total: 2430 unassociated sources

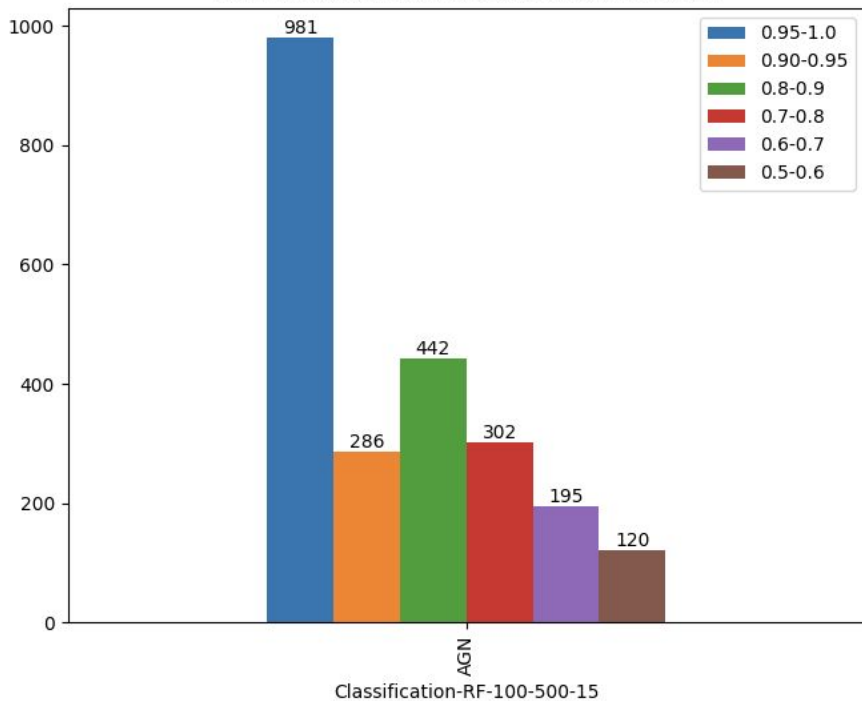


Random Forests

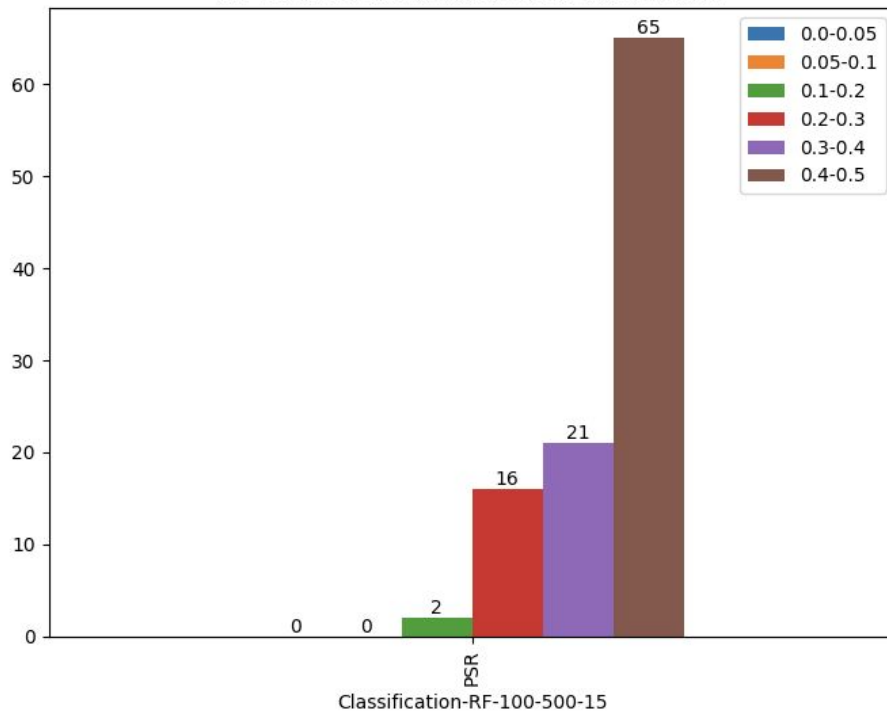
- #trees: 100, # samples: 500, depth: 15

Total: 2430 unassociated sources

2326 unassociated sources AGN classification



104 unassociated sources PSR classification



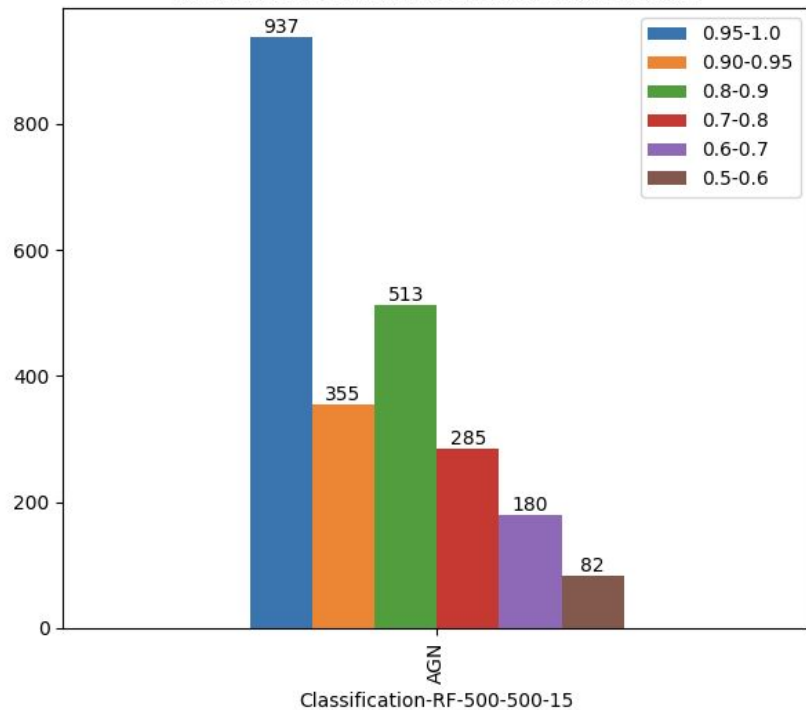
[illegible]

Random Forests

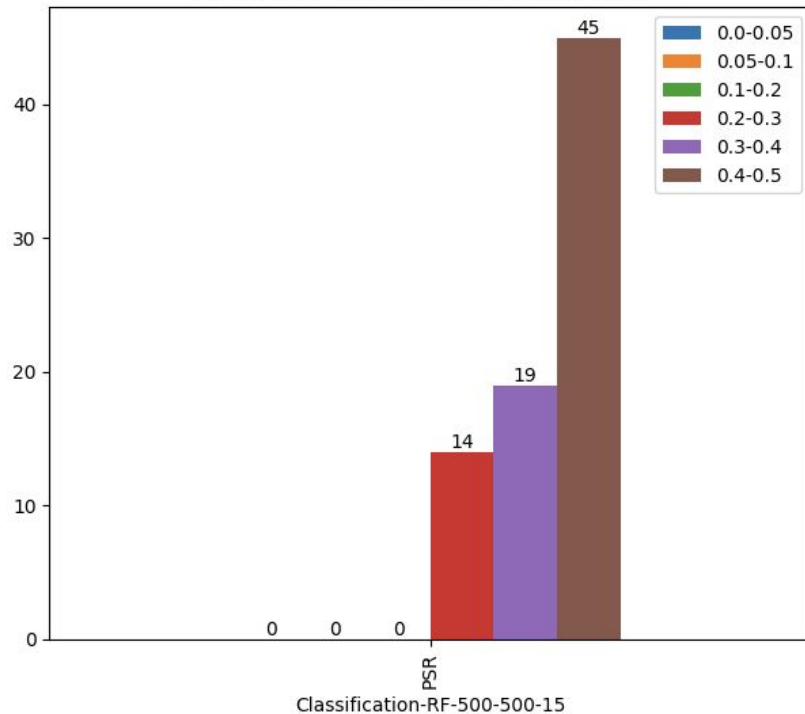
- #trees: 500, # samples: 500, depth: 15

Total: 2430 unassociated sources

2352 unassociated sources AGN classification



78 unassociated sources PSR classification

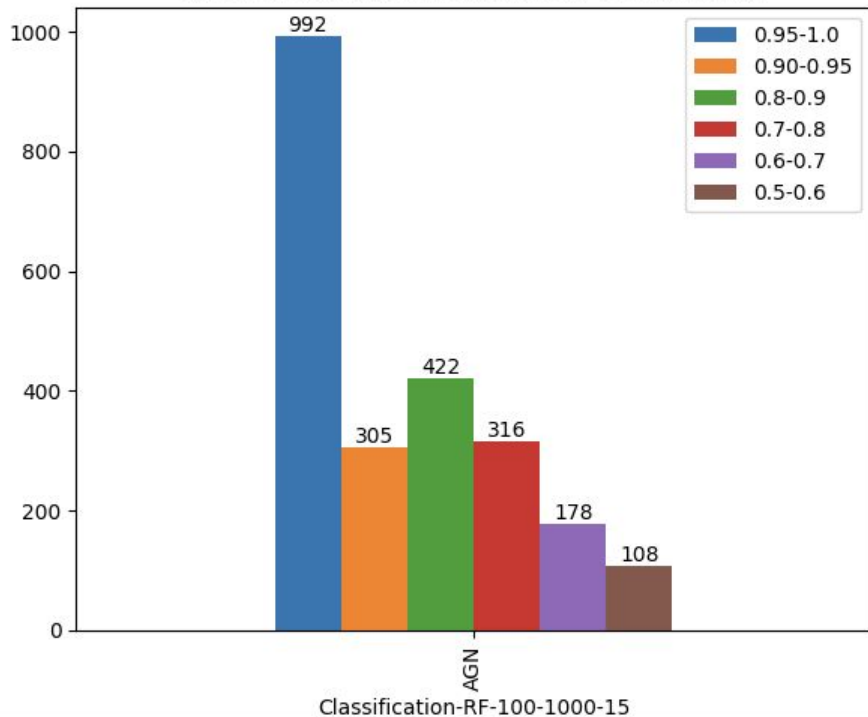


Random Forests

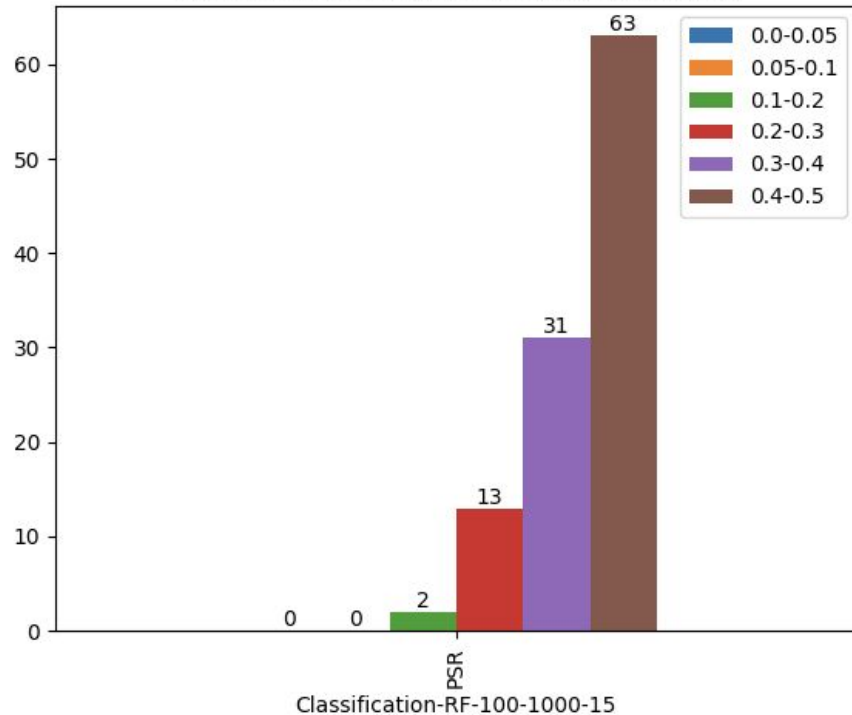
- #trees: 100, # samples: 1000, depth: 15

Total: 2430 unassociated sources

2321 unassociated sources AGN classification



109 unassociated sources PSR classification

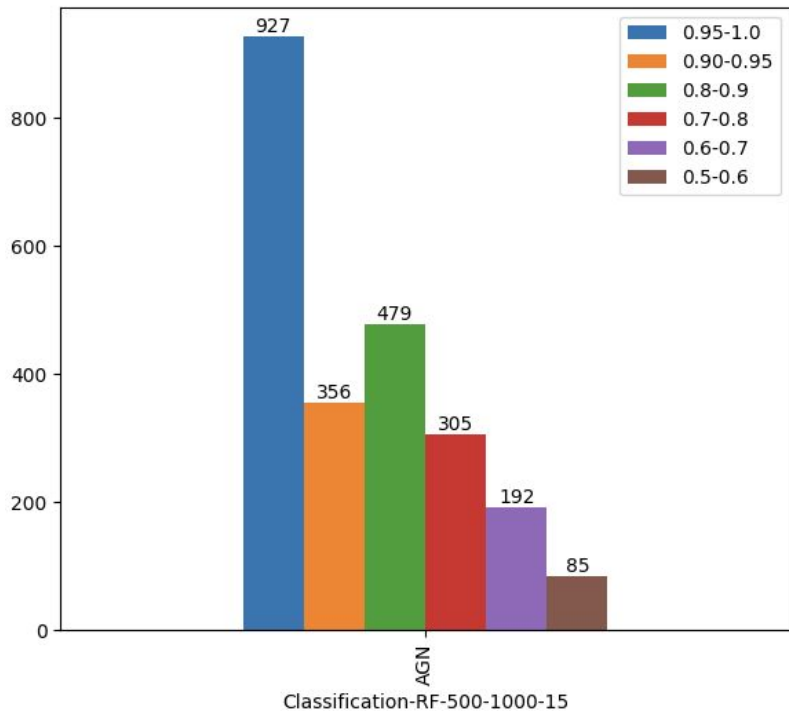


Random Forests

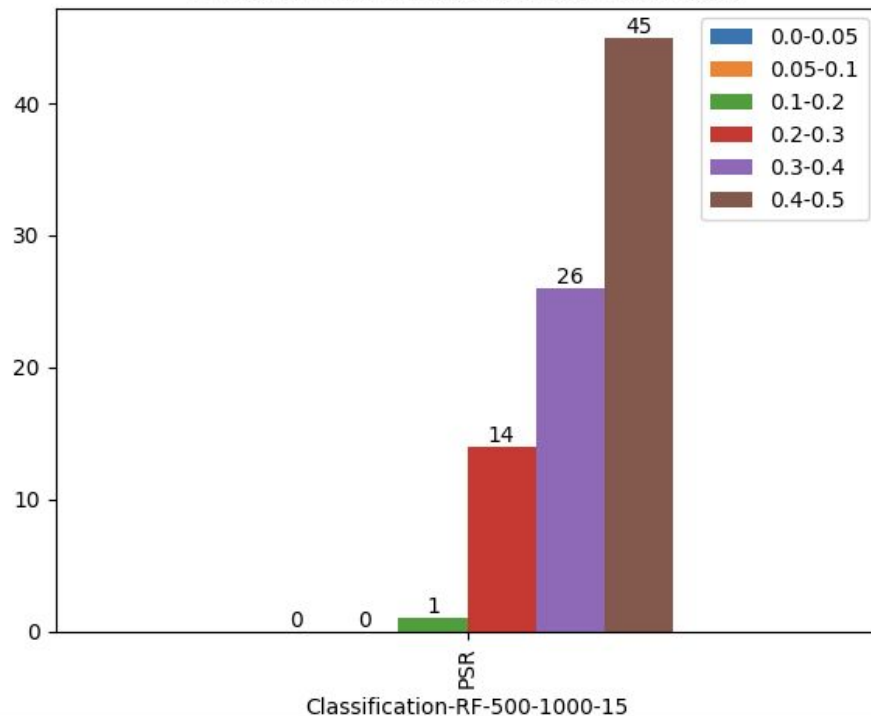
- #trees: 500, # samples: 1000, depth: 15

Total: 2430 unassociated sources

2344 unassociated sources AGN classification



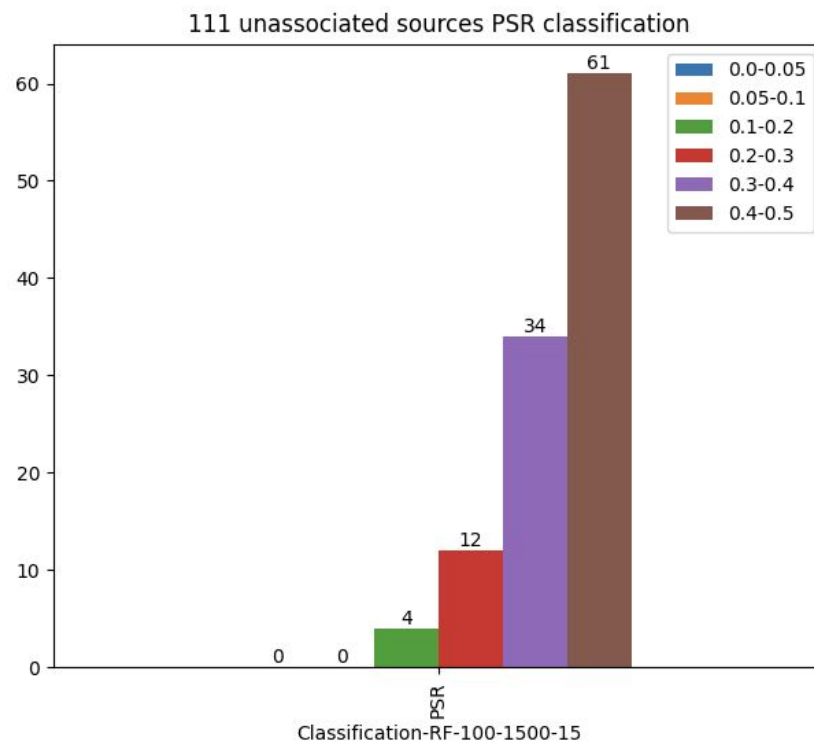
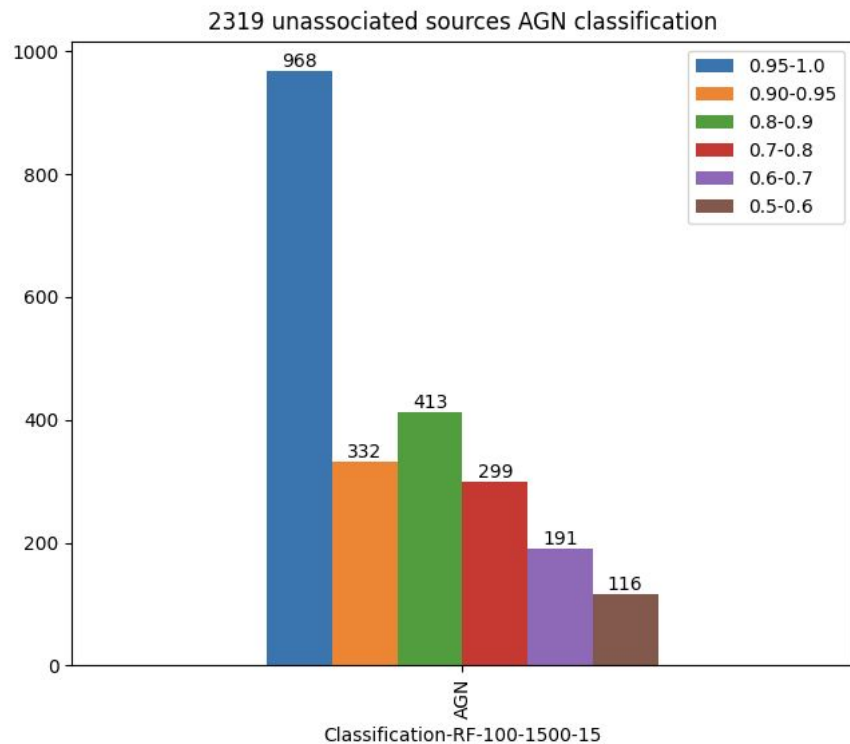
86 unassociated sources PSR classification



Random Forests

- #trees: 100, # samples: 1500, depth: 15

Total: 2430 unassociated sources

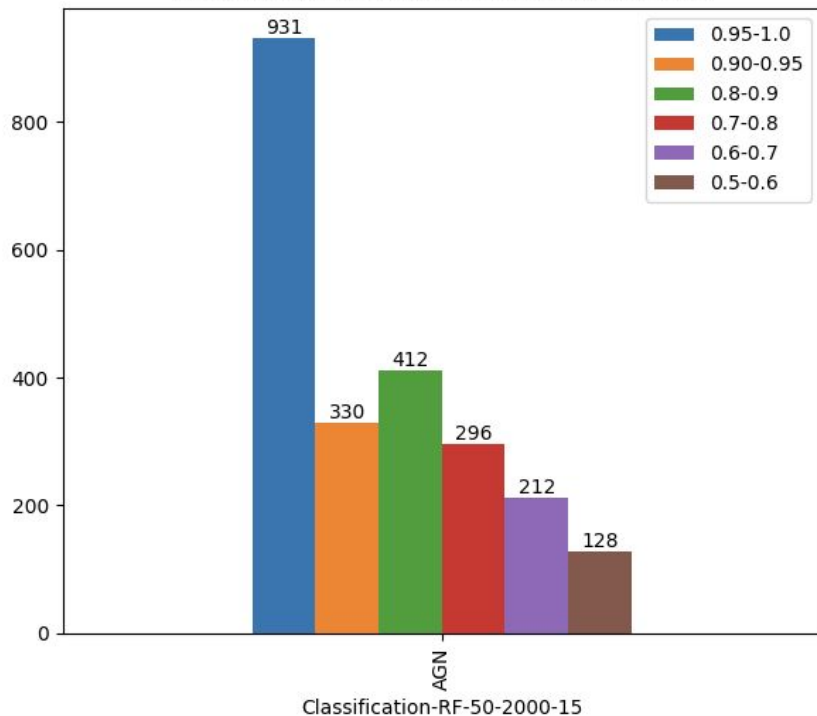


Random Forests

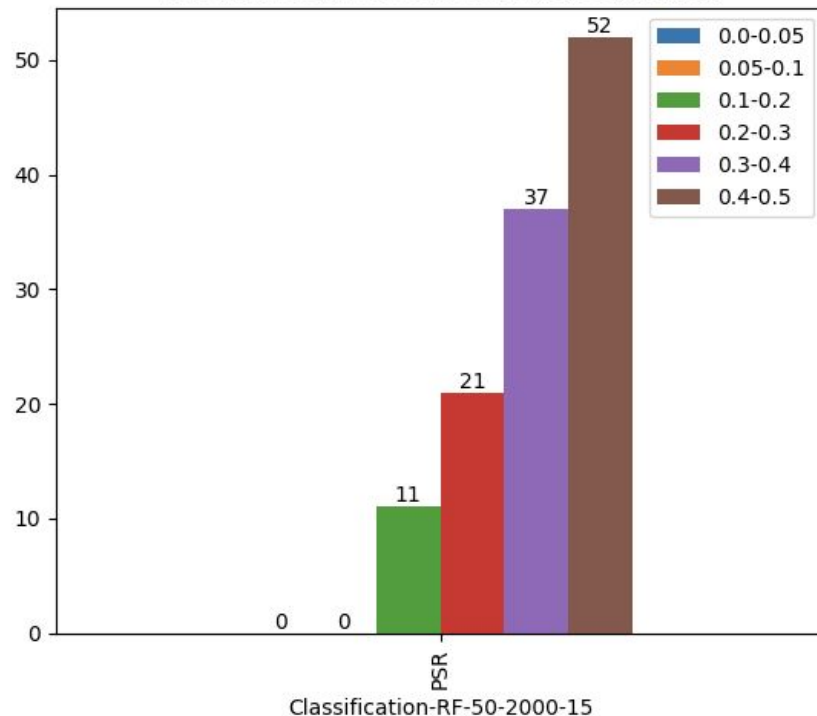
- #trees: 50, # samples: 2000, depth: 15

Total: 2430 unassociated sources

2309 unassociated sources AGN classification



121 unassociated sources PSR classification



Results

- LR:
 - Firmly predicted 1433-1465 AGNs in [0.95-1.0] range
 - Firmly predicted 58-65 PSRs in [0.0-0.05] range until 200 iterations.
- Neural Networks
 - Firmly predicted 1465-1543 AGNs [0.95-1.0] range
 - Firmly predicted 47-119 PSRs [0.0-0.05] range
- Deep Neural Networks
 - Firmly predicted 1464-1927 AGNs [0.95-1.0] range
 - Firmly predicted 37-254 PSRs [0.0-0.05] range
 - More iterations predicted more AGNs in [0.95-1.0] range and more PSRs in [0.0-0.05] range
- Random Forests
 - Firmly predicted 927-992 AGNs [0.95-1.0] range
 - Firmly predicted 0 PSRs in the [0.0-0.05] range
 - Predicted 0-4 PSRs in the [0.5-0.10] range
 - Weakly predicted 45-65 PSRs in [0.45-0.5] range